Laboratorul 7

Introducere in Conversia Analog-Digitală şi Prelucrarea Numerică: Utilizarea XADC-urilor şi a Împărţirii cu Refacerea Restului Partial pe Zynq

7.1.Introducere

Într-o gamă vastă de aplicaţii integrate (embedded), procesarea semnalelor analogice joacă un rol crucial. Ţinând cont că multe dintre informaţiile prelucrate de sisteme electronice provin din lumea reală, sub formă de semnale analogice, pentru ca aceste semnale să fie utilizate eficient în domeniul digital, ele trebuie convertite în date numerice. Acest proces de conversie analog-digitală permite dispozitivelor să interpreteze semnale precum tensiunea de la un senzor sau variaţiile unui semnal electric. Un **semnal analogic** este o reprezentare continuă a unei valori fizice, care poate varia între două limite. Aceasta înseamnă că un semnal analogic poate lua o gamă nelimitată de valori, fiind capabil să reflecte variaţii fine şi detalii subtile ale unui fenomen. Exemple comune de semnale analogice includ tensiunea electrică, sunetul, temperatura sau presiunea. Pe de altă parte, un **semnal digital** este o reprezentare discretă a unei informaţii, care utilizează valori finite sau specifice pentru a reprezenta datele. Semnalele digitale sunt caracterizate prin faptul că pot avea doar un număr limitat de valori, cum ar fi 0 şi 1, care corespund stărilor de "OFF" şi "ON". Aceste semnale sunt mai rezistente la zgomot şi interferenţe, ceea ce le face ideale pentru prelucrarea şi stocarea informaţiei în computere şi alte sisteme digitale.

Una dintre soluţiile eficiente pentru conversia semnalelor analogice în format digital, care poate fi prelucrat de un procesor sau un microcontroler, este utilizarea Convertorului Analog-Digital (ADC). Platforma Zynq, dezvoltată de Xilinx, integrează ADC-uri, cunoscute sub numele de Xilinx Analog to Digital Converter sau XADC-uri, care facilitează conversia semnalelor analogice într-un format digital, utilizând un proces de conversie de tip succesiv aproximativ, la o rata de eşantionare 1 milion de eşantioane pe secunda(**1 MSPS**). Aceste XADC-uri permit citirea semnalelor analogice cu o rezoluţie de **12 biţi**, fiind capabile să gestioneze tensiuni de intrare **diferenţiale** de **maxim 1 volt,** ceea ce îmbunătăţeşte precizia şi robusteţea măsurătorilor prin reducerea zgomotului. Prezenta intrărilor diferenţiale in Zynq-7000 presupune că semnalul de intrare este măsurat între doua referinţe (spre exemplu porturile VP şi VN a unui pin analogic), nu faţă de un potenţial comun (masă). Această abordare diferenţială oferă o imunitate ridicată la zgomot şi permite măsurători mult mai precise, fiind ideală pentru aplicaţii de înaltă robusteţe în medii zgomotoase.

Conversia analogică de tip succesiv aproximativ implică mai mulţi paşi esenţiali pentru a transforma un semnal analogic într-un cod digital. În primul rând, semnalul analogic este eşantionat la intervale fixe printr-un circuit de sample and hold, care menţine valoarea semnalului pentru a permite ADC-ului să efectueze conversia. Apoi, procesul de conversie începe prin compararea tensiunii eşantionate cu o tensiune de referinţă. Algoritmul de conversie succesivă utilizează un registru de comparaţie şi o reţea de rezistente pentru a determina nivelul tensiunii analogice, făcând o presupunere iniţială despre valoarea analogică, care este rafinată pas cu pas, reducând intervalul de căutare. Pe baza rezultatelor comparaţiei, ADC-ul generează un număr binar pe un număr fix de biţi dat de rezoluţia convertorului (de exemplu in cazul Zybo rezoluţia e 12 biţi) pentru a reprezenta valoarea analogică.

Pe lângă măsurarea semnalelor externe, XADC-urile de pe Zynq pot monitoriza si parametrii interni critici, fără a necesita circuite suplimentare. Acest lucru aduce avantaje din punctul de vedere al eficientei energetice, robusteţii aplicaţiei si a costurilor de implementare.

Una dintre mărimile esenţiale care pot fi măsurate este **temperatura die-ului**, care reprezintă temperatura internă a cipului. Monitorizarea acestei temperaturi este crucială pentru prevenirea supraîncălzirii, care poate afecta performanţa şi durabilitatea cipului. Dacă temperatura depăşeşte limitele specificate, poate duce la un fenomen numit cpu throttling (reducerea performanţei cpu-ului) sau chiar la deteriorarea permanentă a circuitului.

Un alt parametru important este VCCINT, care reprezintă tensiunea utilizată pentru alimentarea componentelor interne ale sistemului de procesare din Zynq. Această tensiune trebuie să fie menţinută în limitele specificate pentru a asigura o funcţionare corectă a procesorului. Măsurarea VCCINT ajută la detectarea problemelor de alimentare şi la asigurarea stabilităţii sistemului.

VCCAUX reprezintă tensiunea de alimentare auxiliară, care alimentează circuitele auxiliare ale sistemului, cum ar fi logica de control şi interfeţele externe. Este importantă pentru funcţionarea componentelor secundare, care deşi nu sunt critice, contribuie la funcţionarea eficientă a resurselor sistemului. Monitorizarea acestei tensiuni asigură că circuitele auxiliare funcţionează optim. Parametrul analogic măsurat VBRAM, este tensiunea necesară pentru alimentarea memoriei RAM din Zynq. Stabilitatea acestei tensiuni este esenţială pentru operarea corectă a memoriei volatile. Măsurarea VBRAM permite detectarea eventualelor probleme de alimentare care ar putea afecta performanţa memoriei.

VCCPINT se referă la tensiunea de alimentare pentru procesele interne ale cipului, care trebuie să fie menţinută în limitele de operare pentru a asigura funcţionarea corectă a procesorului. Monitorizarea acestei tensiuni ajută la prevenirea erorilor în execuţia instrucţiunilor. În final, VCCPAUX este tensiunea de alimentare pentru circuitele auxiliare, iar VCCPDRO se referă la tensiunea utilizată pentru anumite componente de tip drop (spre exemplu: regulatoare de tensiune lineare, amplificatoare de semnal etc) . Monitorizarea acestor tensiuni este importantă pentru funcţiile non-critice, dar esenţiale, contribuind astfel la stabilitatea şi performanţa sistemului.

Aceste convertoare sunt integrate direct în arhitectura dispozitivului Zynq, beneficiind astfel de legătura strânsă dintre sistemul de procesare (PS - Processing System) şi logica programabilă (PL - Programmable Logic), caracteristică definitorie a acestei familii de cipuri. In diagrama din Figura 7.1, preluata din [1] este ilustrata arhitectura internă a XADC-ului (Xilinx Analog to Digital Converter) utilizat în FPGA-urile Zynq. Ea evidenţiază fluxul semnalelor analogice şi modul în care acestea sunt convertite în semnale digitale.

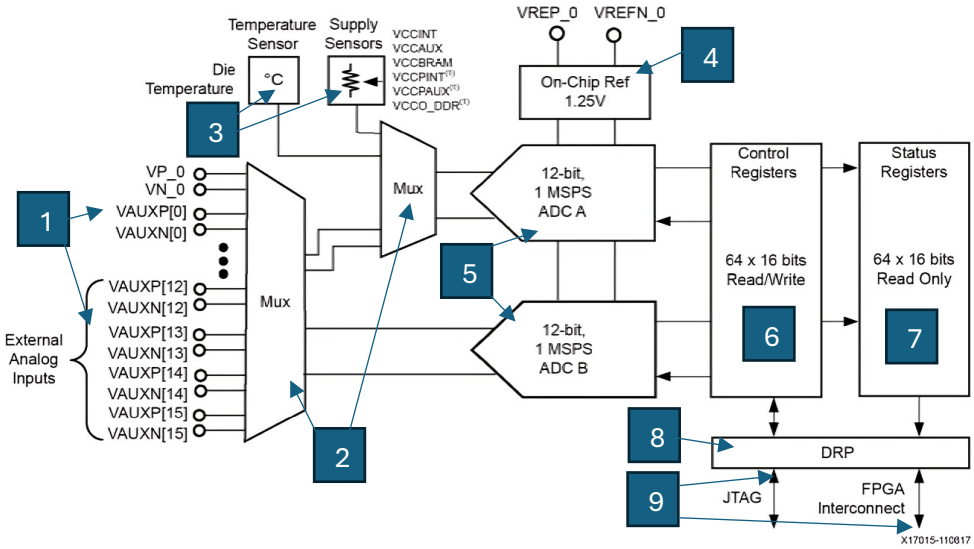

Figura 7.1 Diagrama bloc interna a unui XADC [1]

Pentru a explica ce semnifica fiecare componenta din diagrama bloc prezentata in Figura 7.1. s-au asignat numere corespunzătoare fiecărui modul. De asemenea, in Tabelul VII.1 sunt enumerate denumirile fiecărei componente si semnificaţia acestora.

Tabelul VII.1 Semnificaţia elementelor numerotate Figura 7.1

Nr.	Nume	Semnificaţie sau Functie
1	**External Analog Inputs**	VP_0 şi VN_0 sunt intrări diferenţiale pentru canalul 0, adică preiau un semnal diferenţial, unde VP este pozitiv şi VN este negativ. De regula VP_0, VN_0 sunt folosite la canalele principale ale XADC-ului, dedicate pentru măsurători diferenţiale critice ale tensiunilor externe. Intrările numerotate (de la VAUXP[0] şi VAUXN[0] până la VAUXP[15] şi VAUXN[15]) reprezintă alte canale analogice auxiliare. Pot fi utilizate pentru a măsura semnale de la senzori externi conectaţi la aceste intrări.
2	**Multiplexoare**	Un multiplexor este folosit pentru a selecta ce pereche de intrări va fi trimisă către convertorul ADC. Multiplexoarele comută între diferitele intrări analogice (inclusiv intrările auxiliare VAUX şi intrările pentru temperatură sau senzori de tensiune).
3	**Senzori interni (Temperature Sensor şi Supply Sensors)**	Acest bloc reprezintă senzorii integraţi în XADC, care măsoară temperatura internă a cipului şi tensiunile de alimentare interne ale FPGA-ului (VCCINT, VCCAUX, etc.). Aceste măsurători sunt utile pentru monitorizarea stării dispozitivului şi a condiţiilor de funcţionare.
4	**On-Chip Reference**	XADC-ul foloseşte o referinţă internă de tensiune de 1.25V (VREFP şi VREFN) pentru a asigura acurateţea conversiilor analog-digitale. Această referinţă stabileşte limitele tensiunii pentru semnalele analogice care sunt măsurate
5	**12-bit ADCs (ADC A şi ADC B)**	XADC-ul are două convertoare analog-digitale (ADC-uri), fiecare cu o rezoluţie de 12 biţi şi o rată de eşantionare de 1 MSPS (1 milion de eşantioane pe secundă) Aceste convertoare primesc semnalul analogic de la multiplexoare şi îl transformă într-o valoare digitală de 12 biţi, pe baza referinţei de tensiune şi a valorilor măsurate
6	**Control Registers**	După ce semnalul a fost convertit în format digital, datele sunt stocate în registre de control care pot fi citite sau scrise de utilizator. Aceste registre conţin informaţii despre configurarea şi starea conversiei, permiţând configurarea XADC-ului pentru diferite tipuri de măsurători şi intrări.
7	**Status Registers**	În aceste registre sunt stocate informaţii despre starea actuală a XADC-ului, cum ar fi valorile curente măsurate şi datele despre sănătatea cipului (temperatură, tensiune, etc.). Acestea sunt doar pentru citire
8	**Dynamic Reconfiguration Port (DRP)**	DRP-ul permite configurarea dinamică a XADC-ului în timpul funcţionării, adică poţi schimba setările fără a opri FPGA-ul. Aceasta este o caracteristică puternică, permiţând schimbarea rapidă între diferite tipuri de măsurători
9	**Interfaţă JTAG şi FPGA**	JTAG-ul este o interfaţă standard pentru testare şi depanare, iar interfaţa FPGA permite accesul la datele măsurate şi controlul funcţionării XADC-ului din logica FPGA.

Fluxul procesării unui semnal folosind un XADC de pe Zynq începe cu preluarea intrării analogice, de exemplu de la pini precum VP_0 şi VN_0, care este selectată de un multiplexor şi

trimisă către unul dintre cele două convertoare ADC de 12 biţi (ADC A sau ADC B). După ce semnalul analogic este convertit într-o valoare digitală, această valoare este scrisă în registrele interne. Accesul la datele convertite, precum şi configurarea XADC-ului, se poate realiza prin intermediul FPGA-ului sau prin interfaţa JTAG. De regulă, senzorii analogici produc semnale într-un interval de tensiuni sau curenţi care trebuie ajustat pentru a reflecta corect unităţile fizice măsurate. Scalarea permite transformarea acestor valori brute acaparate de la senzori într-o formă care să poată fi interpretată corect şi utilizată în aplicaţii practice. O soluţie pentru a scala valorile este împărţirea la un anumit factor de divizare. Pentru a realiza acest lucru vom implementa in FPGA un algoritm de împărţire denumit si împărţirea cu refacerea restului parţial.

7.2. Algoritmi de Împărţire Binara

7.2.1 Împărţirea numerelor binare cu refacerea restului parţial

Algoritmul de împărţire cu refacerea restului parţial este folosit pentru împărţirea numerelor binare fără semn în sistemele digitale. Procesul implică o serie de paşi bine definiţi, care duc la un calcul precis al câtului şi restului. In fiecare etapa a algoritmului se efectuează o scădere a împărţitorului din restul parţial. Daca restul parţial este mai mic decât împărţitorul, adică din operaţia de scădere rezulta un număr negativ, in următorul pas se va reface restul parţial la valoarea anterioara prin adunarea împărţitorului la restul parţial. Aceasta metoda este utilizata atât pentru eficienţa sa cat si pentru simplitatea implementării în hardware, fiind esenţiala pentru rezolvarea corecta a operaţiilor matematice în sistemele de calcul. Având in vedere ca operaţia de împărţire este utilizata in multe procese computaţionale, pentru a rezolva divizarea a doua numere s-au implementat de-a lungul timpului mulţi algoritmi de împărţire binara[2] fiecare având avantajele si dezavantajele lor. Cu toate acestea datorita simplităţii de implementare hardware in aceasta secţiune ne vom focusa asupra algoritmului cu refacerea restului parţial. In cazul in care operanzii utilizaţi in metoda de împărţire cu refacerea restului parţial nu sunt reprezentaţi folosind reprezentarea semn si magnitudine, aceştia trebuie convertiţi in aceasta reprezentare înainte de a aplica procedura de divizare.

Schema bloc a circuitului de împărţire care utilizează algoritmul de divizare bazat pe refacerea restului parţial este ilustrata in Figura 7.2

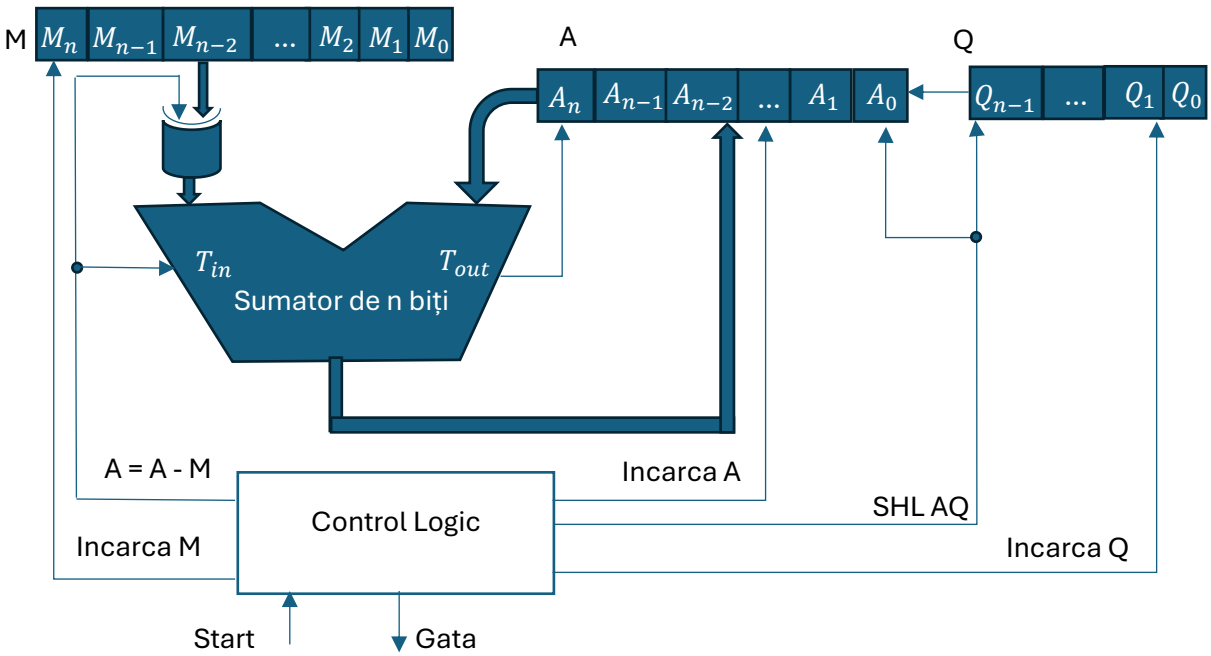

Figura 7.2 Diagrama bloc a modulului de împărţire cu refacerea restului parţial.

Componentele principale ale diagramei 7.2 sunt:

1. **M (Divizor)**: Această unitate stochează divizorul M în timpul operației de împărțire. La începutul procesului, valoarea divizorului este încărcată în circuit pentru a fi folosită în operațiile ulterioare. Daca datele noastre sunt reprezentate pe n biți, numărul de biți a lui M e n + 1.
2. **A(acumulatorul)** Registrul A stochează restul parțial la fiecare etapă a împărțirii. Inițial, acest registru este setat la zero. În timpul procesului de împărțire, A este actualizat prin operația de scădere (A = A - M) și este deplasat(shiftat) pentru a adăuga noi biți din registrul Q. Si in cazul lui A, numărul de biți a lui A va fi n+1.
3. **Q (Registru al câtului)**: Acest registru conține câtul rezultat din operația de împărțire. Inițial, valoarea divizibilului (numărătorul) este stocată în acest registru. În timpul fiecărei iterări, bitul cel mai semnificativ (MSB) din registrul Q este mutat în registrul A, și restul biților sunt deplasat spre stânga. Acest registru e pe n biți.
4. **Sumator de n + 1 biți**: Acest bloc efectuează operațiile de adunare și scădere necesare pentru algoritmul de împărțire. Blocul primește ca intrări registrul M si registrul A si la ieșire oferă suma dintre cele doua, iar in cazul in care exista transport in urma operației de adunare, setează pe 1 bitul T_{out}. De asemenea, in cazul in care valoarea lui M este negativa, operația de transformare in complement fata de 2 se realizează cu ajutorul porții xor, care face operația xor logic pe n + 1 biți, intre biții divizorului M si n+1 biți de 1 si setează transportul de intrare al sumatorului pe 1 (aduna 1 la valoarea negata a lui M pentru a finaliza transformarea in complement fata de 2).
5. **Control Logic (Unitatea de control)**: Această unitate controlează întregul proces de împărțire. Ea gestionează încărcarea valorilor inițiale în registrele A, M și Q, precum și realizarea operațiilor de deplasare și scădere. De asemenea, gestionează condiția de terminare a algoritmului (semnalul Gata), care indică faptul că împărțirea s-a încheiat
6. **Deplasare la stânga (SHL AQ)**: Acest bloc realizează operația de deplasare la stânga a registrelor A și Q. Aceasta este o parte importantă a algoritmului, deoarece după fiecare scădere, rezultatul este deplasat la stânga pentru a pregăti următoarea etapă de împărțire.
7. **Start și Gata**: Aceste semnale controlează inițierea și finalizarea procesului. Semnalul Start inițiază procesul de împărțire, iar Gata indică momentul în care operația s-a finalizat și câtul și restul sunt gata.

Algoritmul de împărțire cu restaurarea restului parțial funcționează prin efectuarea de operații succesive de scădere și restaurare pentru a determina câtul și restul diviziunii între două numere binare. La început, dividendul (numărătorul) este încărcat într-un registru numit Q, iar divizorul(numitorul) este încărcat într-un registru M. Un alt registru, A(acumulator), inițializat cu valoarea zero, va stoca restul parțial în fiecare etapă a procesului. În fiecare pas, conținutul registrelor A și Q este concatenat și deplasat la stânga cu un bit, astfel încât bitul cel mai semnificativ din Q să fie mutat în registrul A. După acest pas, divizorul este scăzut din registrul A. Dacă rezultatul scăderii este pozitiv sau egal cu zero, înseamnă că operația a avut succes și se setează bitul curent (bitul 0) al câtului din Q la 1. Dacă rezultatul este negativ, se restaurează valoarea anterioară a registrului A prin adăugarea divizorului M înapoi și se setează bitul curent din Q la 0. Acest proces se repetă pentru fiecare bit al numărătorului, deplasând și scăzând la fiecare pas. La finalul algoritmului, registrul Q conține câtul rezultat, iar registrul A conține restul. Algoritmul continuă până când toți biții au fost procesați, iar rezultatul diviziunii este determinat. In Figura 7.3 este ilustrata diagrama de tranziții unde fiecare pas necesar realizării împărțirii este inclus. De asemenea, pentru o mai buna înțelegere a funcționarii algoritmului, acesta este trasat pas cu pas pentru o pereche de numere binare, trasarea fiind ilustrata in tabelul VII.2.
Numerele pentru care se face trasarea împărțirii sunt 14 si 3, iar răspunsul algoritmului ar trebui sa fie 4 si 2.

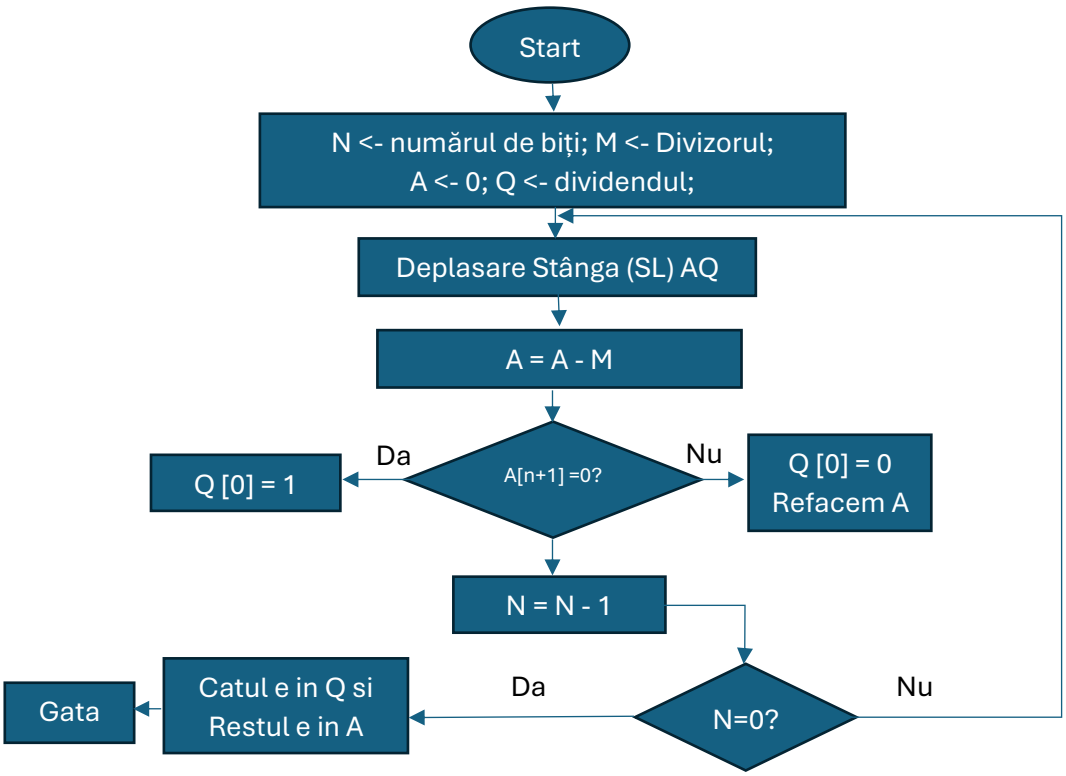

Figura 7.3. Diagrama de tranziție a algoritmului de împărțire cu refacerea restului parțial

In Tabelul VII.2 de mai jos este prezentata trasarea pas cu pas a algoritmului de împărțire bazat pe metoda cu refacerea restului parţial.

Tabelul VII.2. Trasarea pas cu pas a algoritmului de împărţire cu refacerea restului parţial

N	M(divizorul)	A	Q (dividentul)	Operaţia Executata
4	00011	00000	1110	Iniţializare
		00001	110?	SL AQ
		11110	110?	A = A – M (pe -M îl obţinem convertind pe M in complement fata de 2, iar apoi operaţia de scădere devine o operaţie de adunare.)
		00001	1100	Daca cel mai semnificativ bit a lui A e 1, adică rezultatul este negativ, atunci bitul Q[0] = 0 si refacem acumulatorul A
3		00011	100?	SL AQ
		00000	100?	A = A - M
		00000	1001	Cel mai semnificativ bit a lui A este 0, adică rezultatul este >= 0, atunci bitul Q[0]=1
2		00001	001?	SL AQ
		11110	001?	A = A - M
		00001	0010	Bitul cel mai semnificativ a lui A este 1, atunci bitul Q[0] = 0 si refacem A
1		00010	010?	SL AQ
		11111	010?	A = A - M
		00010	0100	Cel mai semnificativ bit a lui A este 1 atunci bitul Q[0] devine 0 si refacem A.
0				Daca am ajuns la n = 0 atunci am finalizat operaţia si catul nostru se găseşte in Q, iar restul e in A.

7.2.2 Împărţirea numerelor binare fără refacerea restului parţial

Operaţia de refacere a restului parţial apare doar atunci când scăderea împărţitorului din restul parţial generează un rezultat negativ. În cazul în care acest lucru se întâmplă frecvent, timpul de execuţie al operaţiei de împărţire creşte. Prin urmare, eliminarea nevoii de refacere a restului parţial poate duce la îmbunătăţirea performanţei împărţirii. În algoritmul standard de împărţire cu refacerea restului parţial, se scade împărţitorul din rest (A = A - M). Dacă rezultatul este negativ, restul parţial se readuce la valoarea anterioară (A = A + M), urmat de deplasarea la stânga, operaţie echivalenta cu înmulţirea cu 2 (A = A * 2). O alternativă ar fi să nu refacem imediat restul parţial dacă rezultatul este negativ, ci să continuăm cu deplasarea la stânga şi aplicarea corecţiilor ulterior, de exemplu (A = 2A - 2M + M), ceea ce poate reduce numărul de operaţii. Aşadar, in fiecare pas al operaţiei de împărţire folosind metoda fără refacerea restului parţial, se verifica mai întâi semnul acumulatorului A, daca semnul este 0, adică acumulatorul este pozitiv, se realizează operaţia de deplasare la stânga urmata de o operaţie de scădere (A = A – M), daca semnul este 1, acumulatorul este negativ, se realizează operaţia de deplasare la stânga urmata de o operaţie de adunare (A = A + M). După finalizarea iteraţiilor, in ultimul pas, se verifica valoarea acumulatorului A, unde este stocat restul. Daca acesta valoare este negativa, se realizează o corecţie adunând la valoarea acumulatorului, valoarea divizorului (A = A + M), altfel rezultatul este corect si algoritmul de împărţire se termina. Diagrama care prezinta toţi paşii necesari realizării algoritmului de împărţire fără refacerea restului parţial este ilustrata in Figura 7.4.

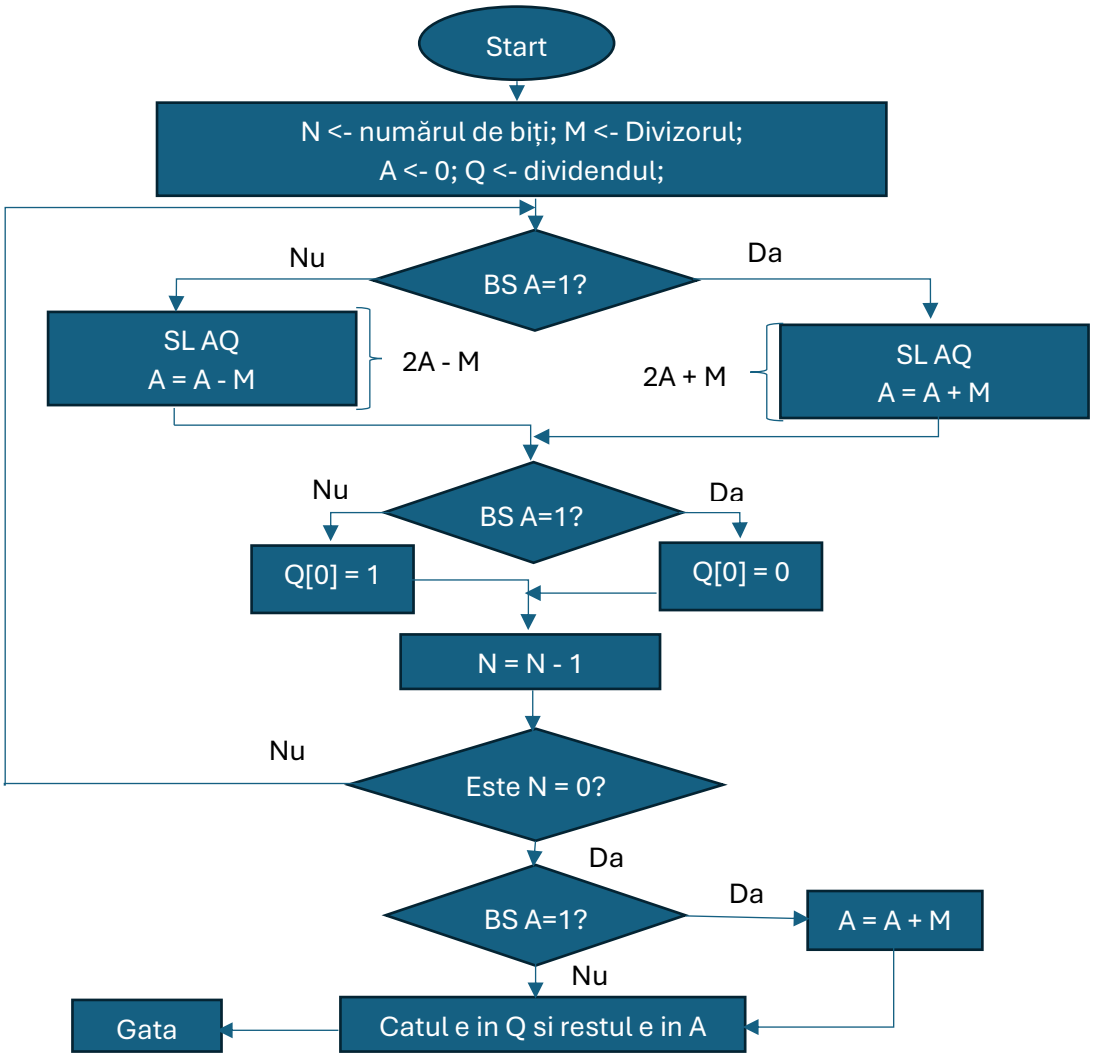

Figura 7.4. Diagrama de tranziţie a algoritmului de împărţire fără refacerea restului parţial

De asemenea, pentru a înţelege mai bine modul de funcţionare al algoritmului, vom trasa pas cu pas metoda de împărţire fără refacerea restului parţial, pe exemplul prezentat si in secţiunea anterioara, adică 14 împărţit la 3. Aceasta trasare este prezentata in Tabelul VII.3. Semnificaţia elementelor din antetul tabelului rămâne aceeaşi ca si in exemplul precedent. Este important de menţionat si faptul ca pe lângă creşterea vitezei, algoritmul de împărţire fără refacerea restului parţial mai are avantajul ca poate lucra cu numere in orice reprezentare. Diagrama bloc a metodei de împărţire fără refacerea restului rămâne la fel ca si in cazul precedent, diferenţele fiind realizate in unitatea de control a modulului de împărţire.

Tabelul VII.3. Trasarea pas cu pas a algoritmului de împărţire fără refacerea restului parţial

N	M (divizorul)	A	Q (dividendul)	Operaţia Executata
4	00011	00000	1110	Iniţializare
		00001	110?	Daca bitul de semn a lui A e 0, facem mai întâi SL AQ si apoi in pasul următor A = A – M. Aşadar prima operaţie e SL AQ
		11110	110?	A = A – M (Pe -M îl obţinem ca si in algoritmul precedent realizând complementul fata de 2 a lui M)
		11110	1100	Se verifica din nou bitul de semn a lui A. Daca bitul de semn e 1, ca si in cazul nostru Q[0] = 0
3		11101	100?	Ne uitam mai întâi la valoarea bitului de semn a lui A pentru a determina operaţia aritmetica următoare. In cazul nostru este 1, asta înseamnă ca după operaţia curenta de deplasare la stânga vom realiza A = A + M. Realizam operaţia de deplasare la stânga SL AQ.
		00000	100?	A = A + M
		00000	1001	Se verifica bitul de semn a lui A, in cazul nostru este 0, înseamnă ca valoarea lui Q[0] = 1
2		00001	001?	Valoarea bitului de semn a lui A e 0, asta înseamnă ca operaţia următoare va fi de scădere. Executam deplasarea la stânga SL AQ
		11110	001?	A = A - M
		11110	0010	Bitul de semn a lui A e 1 => Q[0] = 0
1		11100	010?	Verificam bitul de semn a lui A. Valoarea acestuia este 1, înseamnă ca operaţia următoare va fi A = A + M. Realizam Operaţia de deplasare la stânga SL AQ
		11111	010?	A = A + M
		11111	0100	Bitul de semn a lui A e 1 => Q[0] = 0
0		00010	0100	Pas de corecţie, in care realizam operaţia A = A + M întrucât semnul lui A rezultat e 1. Acest pas este realizat doar daca valoarea din A in ultima iteraţie a algoritmului rămâne negativa.

7.3. Utilizarea Modulului de Împărțire pentru Procesarea și Scalarea Valorilor Analogice XADC pe Zynq

In aceasta secțiune vom implementa un modul de achiziție de la mai mulți senzori analogici pe platforma Zynq. O parte din senzori sunt prezenți pe placa Zybo, nefiind nevoie de conectare externa, iar pentru a simula achiziția de la un senzor generic extern vom achiziționa date de la un potențiometru. Datele de la potențiometru le vom scala folosind o constanta de scalare pe care o vom seta in partea de PS. Pentru a scala valoarea citita de la un senzor, sau in cazul nostru un potențiometru, vom utiliza algoritmul de împărțire cu refacerea restului parțial. Tensiunile interne ale căror valori le vom măsura sunt cele prezentate si in partea întâi a acestui laborator si anume: temperatura cipului, VCCINT, VCCAUX, VBRAM, VCCPINT, VCCPAUX si VCCPDRO. Semnificația numelor tensiunilor de mai sus a fost descrisa in detaliu in secțiunea 7.1 a acestui capitol. Este important de reamintit faptul ca exemplul prezentat in aceasta secțiune, este unul care își dorește sa ilustreze utilizarea practica a conceptelor si noțiunilor prezentate anterior. Intr-o situație reala abordările prezentate vor trebui adaptate in funcție de problema care se dorește a fi rezolvata.

7.3.1. Implementarea in VHDL a modulului de împărțire cu refacerea restului parțial

Pentru a scala datele provenite de la un senzor sau un potențiometru vom împărții valoarea pe care o achiziționam, la o constanta întreaga, predefinita, fără semn. Întrucât dorim sa realizam împărțirea cat mai eficient, vom accelera hardware algoritmul de împărțire, prin implementarea acestuia in vhdl. Modulul din vhdl va fi apelat din partea de PS, iar când un fanion, pe nume „gata”, ne va indica finalul operației, vom prelua informațiile (din PL in PS) si le vom afișa la consola. Este important de menționat faptul ca, pentru a scala eficient orice valoare provenita de la un anumit senzor, trebuie sa fie consultata foaia tehnica a acelui senzor. In cazul exemplului de fata vom împarți informația achiziționata la o valoare predefinita, pentru a arata cum putem utiliza algoritmul de împărțire cu refacere a restului parțial, accelerat hardware in vhdl. Așadar, creați un proiect nou pentru plăcuta Zybo (cum a fost prezentat in laboratoarele precedente), iar apoi adăugați o componenta noua numita RestoringDivision si adăugați codul de mai jos.

```
library ieee; --includerea librariilor esentiale modulului nostru
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.std_logic_unsigned.all;
-- Declararea entității pentru impartirea cu refacerea restului partial
entity RestoringDivision is
  generic ( N : integer := 12); -- Dimensiunea de referinta a vectorilor de intrare/ieșire
  port (
    clk      : in  std_logic; -- Semnalul de ceas
    rst      : in  std_logic; -- Semnalul de reset
    start    : in  std_logic; -- Semnalul de start pentru inițierea diviziunii
    dividend  : in  std_logic_vector(N-1 downto 0); -- Dividendul (numaratorul)
    divisor   : in  std_logic_vector(N-1 downto 0); -- Divisorul (numitorul)
    quotient  : out std_logic_vector(N-1 downto 0); -- Ieșirea pentru cât
    remainder : out std_logic_vector(N downto 0); -- Ieșirea pentru rest
    gata     : out std_logic -- Semnal care va indica ca operatia e finalizata
  );
end entity RestoringDivision;
```

```
architecture Behavioral of RestoringDivision is
-- Definirea tipului de stare pentru FSM, FSM-ul nostrum are 3 stari
  type state_type is (S_ASTEPTARE, S_CALCUL, S_FINAL);
  signal state  : state_type; -- semnal pentru stocarea starii curente
  signal A     : std_logic_vector(N downto 0); -- semnalul unde vom stoca restul partial
  signal M     : std_logic_vector(N downto 0); -- semnal unde stocam divizorul nostru
  signal Q     : std_logic_vector(N-1 downto 0); -- Q e catul pe N biți
  signal contor : integer range 0 to N; -- contor pentru numărul de iterații
begin
  process(clk, rst)
    variable A_prim, A_partial : std_logic_vector(N downto 0); -- Variabile pentru A
    variable Q_partial :  std_logic_vector(N-1 downto 0);
    variable currentCont : integer range 0 to N;
  begin
    if rst = '1' then -- reseteaza tot
      state    <= S_ASTEPTARE;
      M       <= (others => '0');
      Q       <= (others => '0');
      gata     <= '0';
      contor   <= N;
    elsif rising_edge(clk) then
      case state is  --itereaza prin starile FSM-ului  pentu a rezolva impartirea
        when S_ASTEPTARE =>
          if start = '1' then
            -- incarca divizorul in M si extinde-l la N+1 biti.
            M <= (not('0' & divisor)) + 1; --Transformam M in -M folosind complement fata de 2
            Q <= dividend; --incarcam dividentul
            A <= (others => '0'); -- Initializam A la 0
            contor <= N; -- Initializam contorul la numarul de biti ai operanzilor
            state  <= S_CALCUL;
          end if;
         when S_CALCUL =>
          currentCont := contor;
          Q_partial := Q;
          -- Deplasare A la stanga si concatenare cu MSB a lui Q
         A_prim := A(N-1 downto 0) & Q_partial(N-1);
         Q_partial(N-1 downto 1) := Q_partial(N-2 downto 0);   -- Shift left Q
         -- Salvam valuarea lui A inainte de procesare in caz ca vrem sa-l restauram
          A_partial := A_prim;
          -- Realizam operatia A = A – M, care in complement fata de 2 e A = A + M
          A_prim := A_prim + M;
          if A_prim(N) = '1' then --verificam MSB a lui A daca este 1
            A_prim := A_partial; -- In cazul in care este MSB a lui A e 1 restauram restul nostru
            Q_partial(0) := '0';   -- si setam LSB a lui Q la 0
          else -- altfel
            Q_partial(0) := '1';   -- si setam LSB a lui Q la 1
          end if;
```

```
            -- decrementeaza contorul
            currentCont := currentCont - 1;
            -- Actualizarea semnalelor interne
            A <= A_prim;  -- Actualizarea lui A
            Q <= Q_partial; -- Actualizarea lui Q
            contor <= currentCont; -- Actualizarea contorului
            -- Verificarea dacă calculul este complet
            if currentCont = 0 then
               state <= S_FINAL; -- Schimbarea stării la finalizare
            end if;
          when S_FINAL =>
           -- Ieșirea pentru cât și rest. Stocam
            quotient  <= Q; --iesirea pentru cat
            remainder <= A; --iesirea pentru rest
            gata     <= '1'; --semnalul 1 pentru gata care indica ca e finalizata operatia
            state    <= S_ASTEPTARE; -- Revenirea la starea de aşteptare
        end case;
      end if;
    end process;
  end architecture Behavioral;
```

7.3.2 Proiectarea Hardware in Vivado

In meniul Flow Navigator, in secţiunea IP Integrator, **apasati click** pe "Create Block Design" si introduceţi un nume in fereastra care apare (puteţi lasă si numele care apare implicit adică design_1), apăsaţi apoi pe butonul Ok. Acest proces este ilustrat in Figura 7.5.

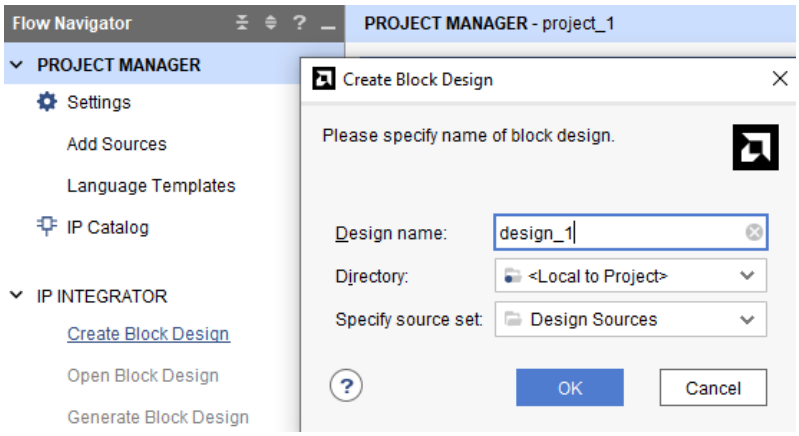

Figura 7.5. Crearea unui nou bloc design.

După crearea noului design, daţi click pe simbolul "**+**" si adăugaţi un bloc Zynq, iar apoi apăsaţi pe "**Run Block Automation**" si pe butonul "**OK**" in fereastra noua care apare. Pentru a utiliza XADC-ul pe Zybo, apăsaţi din nou pe simbolul "**+**" si in fereastra care apare, scrieţi XADC si selectaţi modulul "**XADC Wizard**", similar ca in **Figura 7.6.** Tot in aceasta figura se poate observa si imaginea modulului XADC Wizard, care apare după ce se da click pe opţiunea „XADC Wizard". După adăugarea modulului „XADC Wizard" daţi dublu click pe acesta pentru a-l configura.

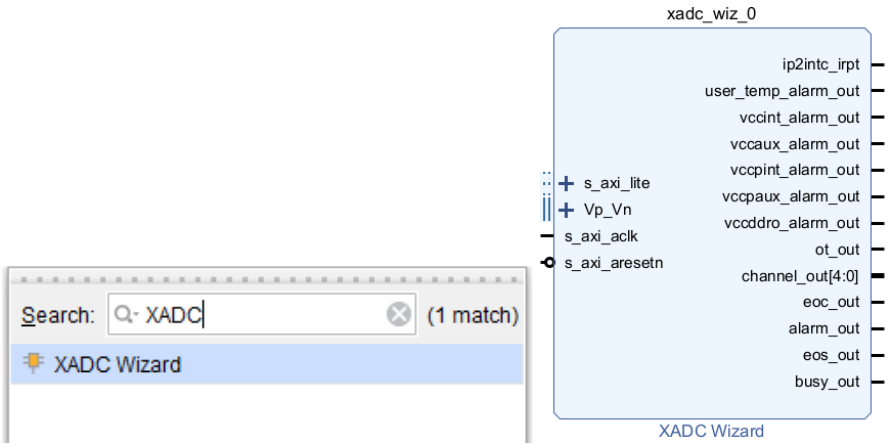

Figura 7.6. Includerea modulului XADC

După ce daţi **dublu click**, configuraţi opţiunile din tabul Basic ca in Figura 7.7. Alegerea opţiunii **AXI4Lite**, este realizata pentru a comunica prin protocolul AXI cu procesorul.

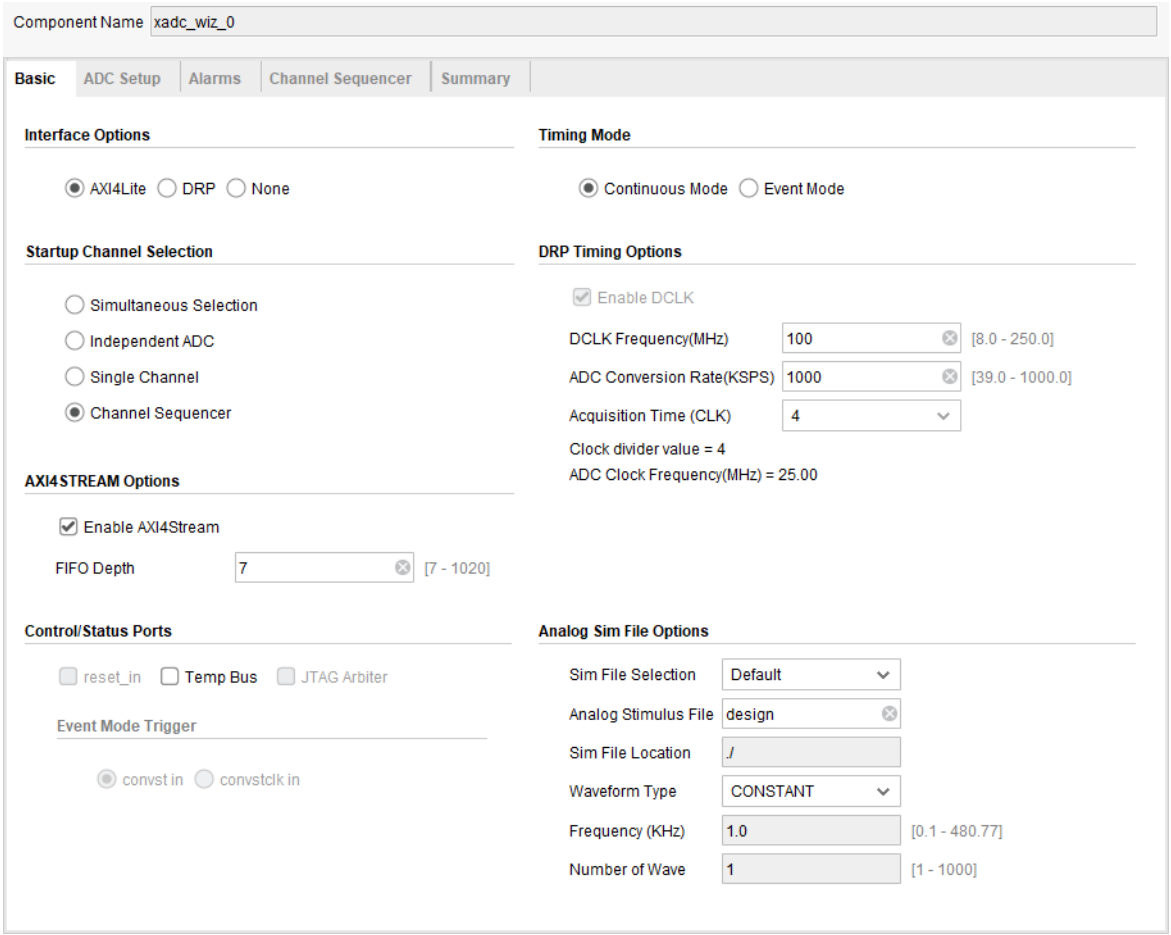

Figura 7.7. Configurarea tabului basic a componentei XADC

Apoi alegem opţiunea „**Channel Sequencer**”, întrucât vom dori sa convertim informaţia de la mai multe canale, aşa cum a fost precizat si anterior, in mod secvenţial. Daca am folosi un singur canal de conversie am selecta „Single Channel”. Opţiunea **“Enable AXI4Stream**” activează modul AXI4Stream pentru transmiterea datelor convertite ADC către procesor. Selectarea opţiunii “**Continuous Mode**” determina ca ADC-ul sa funcţioneze în mod continuu, fără a aştepta un semnal de declanşare. In secţiunea DRP Timing Options, “**Enable DCLK**“ activează utilizarea unui

ceas dedicat pentru conversia ADC. Frecvența ceasului DCLK, **DCLK Frequency(MHz),** in acest exemplu, este setată la 100 MHz (valorile acceptate sunt între 8 şi 250 MHz). Rata de conversie a ADC-ului, **ADC Conversion Rate (KSPS),** măsurat în mii de probe pe secunda este setat la 1000 KSPS (1000 de probe pe secundă). **Acquisition Time (CLK),** este timpul de achiziție, exprimat în cicluri de ceas. **Clock Divider**, împarte frecvența ceasului de intrare pentru a genera ceasul ADC. În exemplul nostru, clock divider-ul este 4, deci ceasul de conversie ADC va fi 25 MHz, deoarece 100 MHz / 4 = 25Mhz. Semnalele **convst_in** şi **convstclk_in** sunt semnale externe care pot declanşa o conversie doar când XADC-ul este configurat sa ruleze în mod Event Mode. In exemplul nostru nu ne va influenta conversia cu nimic întrucât setam modulul sa funcţioneze in mod continuu.

In tabul "**ADC Setup**" vom lăsa configurația implicita. Imaginea cu configurația din acest tab este ilustrata in Figura 7.8.

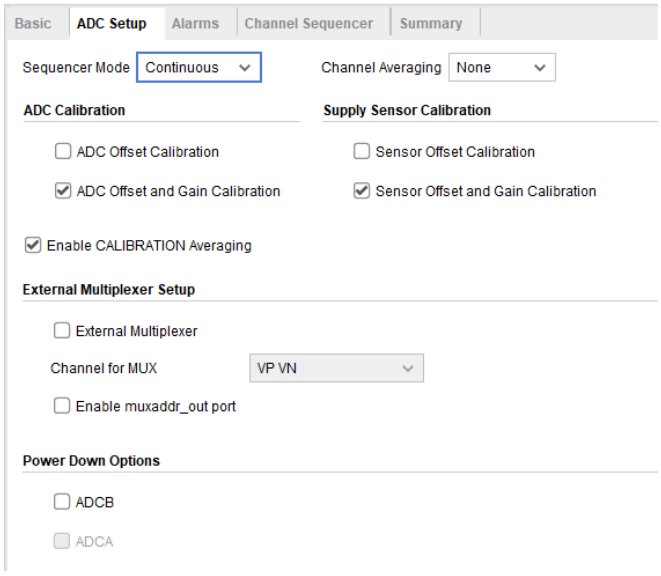

Figura 7.8. Configurarea tabului „ADC Setup"

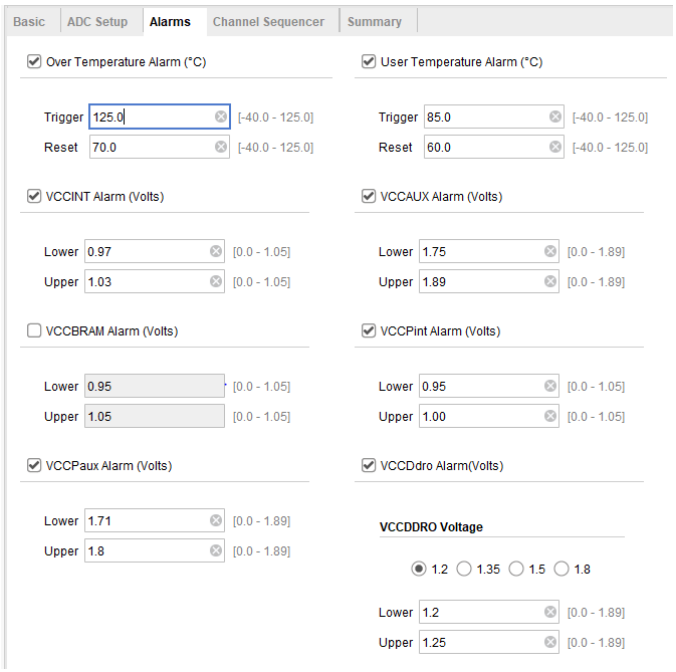

Figura. 7.9. In exemplul nostru configurația tabului „Alarms" rămâne cea implicita

Tabul „**Alarms**” din configurarea XADC în Vivado este dedicat setării alarmelor care pot monitoriza şi semnaliza diferite condiţii critice sau de eroare în timpul funcţionării modulului ADC. Aceste alarme sunt utile pentru monitorizarea unor parametri esenţiali cum ar fi temperatura sau tensiunea, care pot preveni deteriorarea sistemului sau pot semnaliza funcţionări defectuoase. In exemplul nostru, vom lăsa setările implicite pentru tabul ADC Setup, lucru care este ilustrat si in Figura 7.9.

Tabul „**Channel Sequencer**” din XADC permite configurarea secvenţei automate de conversie a mai multor canale analogice. Acesta simplifică monitorizarea achiziţiei de date de la diferiţi senzori permiţând selectarea canalelor dorite, stabilirea ordinii de conversie şi ajustarea timpului de achiziţie pentru fiecare canal. In acest tab vom seta care canale dorim sa le activam. Conform cerinţei enunţate la începutul acestea secţiuni, dorim sa activam VCCINT, VCCAUX, VBRAM, VCCPINT, VCCPAUX si VCCPDRO dar si canalele responsabile pentru achiziţia de la senzori externi conectaţi la plăcuţa Zybo. Deşi noi vom conecta doar un potenţiometru la XADC-uri, care va avea nevoie de un singur canal, noi vom activa toate canalele de achiziţie pentru XADC-ul de pe plăcuţa noastră pentru a ilustra cum se realizează acest lucru.

Basic | ADC Setup | Alarms | **Channel Sequencer** | Summary

	Channel Enable	Average Enable	Bipolar	Acquisition Time
CALIBRATION	☐			
TEMPERATURE	☑	☐		
VCCINT	☑	☐		
VCCAUX	☑	☐		
VCCBRAM	☑	☐		
VCCPINT	☑	☐		
VCCPAUX	☑	☐		
VCCDDRO	☑	☐		
VP/VN	☐	☐	☐	☐
VREFP	☐			
VREFN	☐			
vauxp0/vauxn0	☐	☐	☐	☐
vauxp1/vauxn1	☐	☐	☐	☐
vauxp2/vauxn2	☐	☐	☐	☐
vauxp3/vauxn3	☐	☐	☐	☐
vauxp4/vauxn4	☐	☐	☐	☐
vauxp5/vauxn5	☐	☐	☐	☐
vauxp6/vauxn6	☑	☐	☐	☐
vauxp7/vauxn7	☑	☐	☐	☐
vauxp8/vauxn8	☐	☐	☐	☐
vauxp9/vauxn9	☐	☐	☐	☐
vauxp10/vauxn10	☐	☐	☐	☐
vauxp11/vauxn11	☐	☐	☐	☐
vauxp12/vauxn12	☐	☐	☐	☐
vauxp13/vauxn13	☐	☐	☐	☐
vauxp14/vauxn14	☑	☐	☐	☐
vauxp15/vauxn15	☑	☐	☐	☐

Figura 7.10. Configurarea canalelor XADC-ului conform enunțului problemei.

Aşadar in tabul „**Channel Sequencer**” bifaţi opţiunile care ţin de tensiunile interne si de canalele externe de achiziţie. In Figura 7.10 ilustram canalele care trebuie bifate pentru a obţine rezultatele dorite. Pe placa Zybo 7000, aveţi doar 4 pini de achiziţie externă pentru XADC deoarece aceasta a fost proiectată cu un număr limitat de intrări analogice externe disponibile pe conectorul său XADC. Deşi XADC-ul intern al Zynq-7000 suportă mai multe canale auxiliare, designul hardware al plăcii Zybo expune doar un subset din aceste canale, adică două perechi de pini diferenţiali (Vin_P şi Vin_N) pentru achiziţie analogică. Aceasta este o limitare fizică a plăcii, nu a XADC-ului în sine. Din acest motiv noi selectam doar cele 4 porturi vaux in Figura 7.10. Putem sa identificam care porturi vaux sa le activam uitându-ne in fişierul de constrângeri al plăcii sau pe bareta de pini XADC de pe plăcuta Zybo, ca in Figura 7.11.

Figura 7.11. XADC-urile disponibile de pe portul JA

După configurarea XADC-ului apăsaţi butonul “Run Connection Automation” pentru a integra rapid si pentru a conecta componenta XADC de procesorul Zynq. In fereastra care apare bifaţi toate opţiunile (“**All Automation**”), ca in Figura 7.12, apoi apăsaţi butonul “**Ok**”.

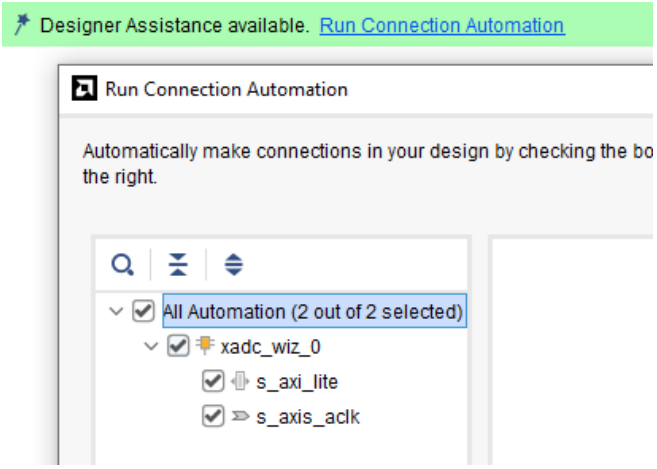

Figura 7.12. Selectarea opţiunilor in fereastra de „Run Connection Automation”

Pentru a conecta corect intrările auxiliare XADC la pinii fizici de pe placa Zybo, astfel încât FPGA-ul să ştie de unde să citească semnalele analogice pentru a le trimite la procesor, aceste intrări vor trebui configurate intr-un fişier de constrângeri XDC. Creaţi un nou fişier de constrângeri in proiectul vostru, apăsând **click dreapta** pe directorul „Constraints”, selectând apoi “**Add Sources**”, bifaţi opţiunea “**Add or create constraints**” si apăsaţi butonul “**Next**”. In fereastra noua, apăsaţi pe butonul “**Create File**” si setaţi un nume pentru fişierul vostru xdc (spre exemplu

puteţi sa numiţi fişierul zybo.xdc). Apăsaţi butonul "**Ok**" iar apoi apăsaţi butonul "**Finish**" pentru a adaugă fişierul de constrângeri la proiectul vostru. In tabul Sources, navigate la fişierul create si daţi **dublu click** pentru a-l deschide. Adăugaţi constrângerile de mai jos in acest fişier.

```
##Pmod Header JA (XADC)
set_property -dict { PACKAGE_PIN N15   IOSTANDARD LVCMOS33 } [get_ports { vaux14p }];
set_property -dict { PACKAGE_PIN L14   IOSTANDARD LVCMOS33 } [get_ports { vaux7p }];
set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { vaux15p }];
set_property -dict { PACKAGE_PIN K14   IOSTANDARD LVCMOS33 } [get_ports { vaux6p }];
set_property -dict { PACKAGE_PIN N16   IOSTANDARD LVCMOS33 } [get_ports { vaux14n }];
set_property -dict { PACKAGE_PIN L15   IOSTANDARD LVCMOS33 } [get_ports { vaux7n }];
set_property -dict { PACKAGE_PIN J16   IOSTANDARD LVCMOS33 } [get_ports { vaux15n }];
set_property -dict { PACKAGE_PIN J14   IOSTANDARD LVCMOS33 } [get_ports { vaux6n }];
```

Puteţi seta acest fişier de constrângeri ca si fişier principal de constrângeri dând click dreapta pe fişier si selectând opţiunea „**Set as Target Constraint File**".

In momentul de fata diagrama pe care aţi realizat-o trebuie sa arate ca si cea din **Figura 7.13**.

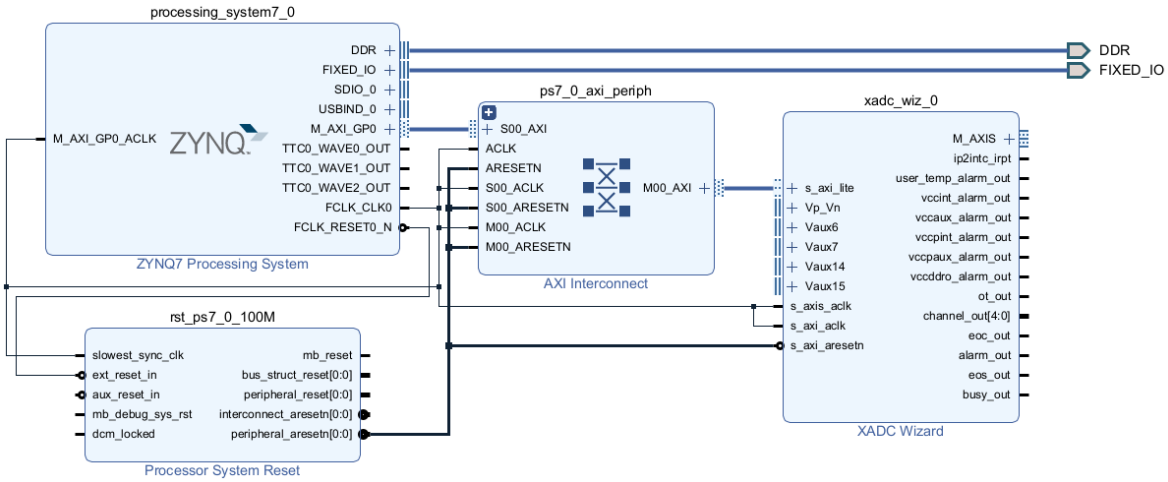

Figura 7.13. Arhitectura intermediara a sistemului care include modulul de achiziţie analogic

Pentru a lega pinii modulului XADC de pinii din fişierul de constrângeri, apăsaţi mai întâi pe butonul sub forma de „+" de pe modulul XADC, de la pinii Vaux6, Vaux7, Vaux14 si Vaux15, pentru a expanda si a face vizibili pinii acestor intrări. Odată expandate intrările se vor vedea cei doi pini diferenţiali de la fiecare intrare. Pentru fiecare pin diferenţial va trebui sa realizaţi următoarea operaţie: dat click stânga pe acel pin pentru a-l selecta (odată selectat îşi va modifica culoarea din negru in portocaliu), daţi apoi click dreapta pe acest pin si din meniul care va apărea selectaţi opţiunea „**Make External**". Aceasta operaţie este ilustrata in Figura 7.14. Alternativ puteţi selecta pinul si sa apăsaţi combinaţia de taste „CTRL+T" pentru a face pinul extern. După realizarea acestei operaţii veţi observa ca apare un mic obiect ca o eticheta, numita "vauxn6_0" ataşata printr-un fir de pinul pentru care am realizat operaţia. Vom redenumi aceasta eticheta cu numele pe care îl avem in fişierul de constrângeri *.xdc si anume vaux6n. Pentru a realiza aceasta redenumire daţi **click dreapta** pe eticheta creata, vauxn6_0, si selectaţi opţiunea "**External Port Properties**", iar la tabul general din acest meniu la secţiunea de nume introduceţi noul nume, vaux6n, si apăsaţi tasta „**Enter**". Veţi observa in panelul de proiectare ca eticheta îşi schimba numele din vauxn6_0 in vaux6n. Aceasta secvenţa de operaţii trebuie realizata pentru toţi pinii diferenţiali auxiliari din XADC ramaşi. Etichetele trebuie sa fie numite intr-un mod adecvat astfel încât sa se poată realiza conexiunea intre pinii xadc-ului si fişierul de constrângeri *.xdc si anume

vauxp6 a cărui eticheta trebuie sa se numească **vaux6p**, **vauxn7** cu eticheta cu numele **vaux7n**, **vauxp7** a cărui eticheta trebuie sa aibă numele **vaux7p**, si aşa mai departe conform *.xdc-ului.

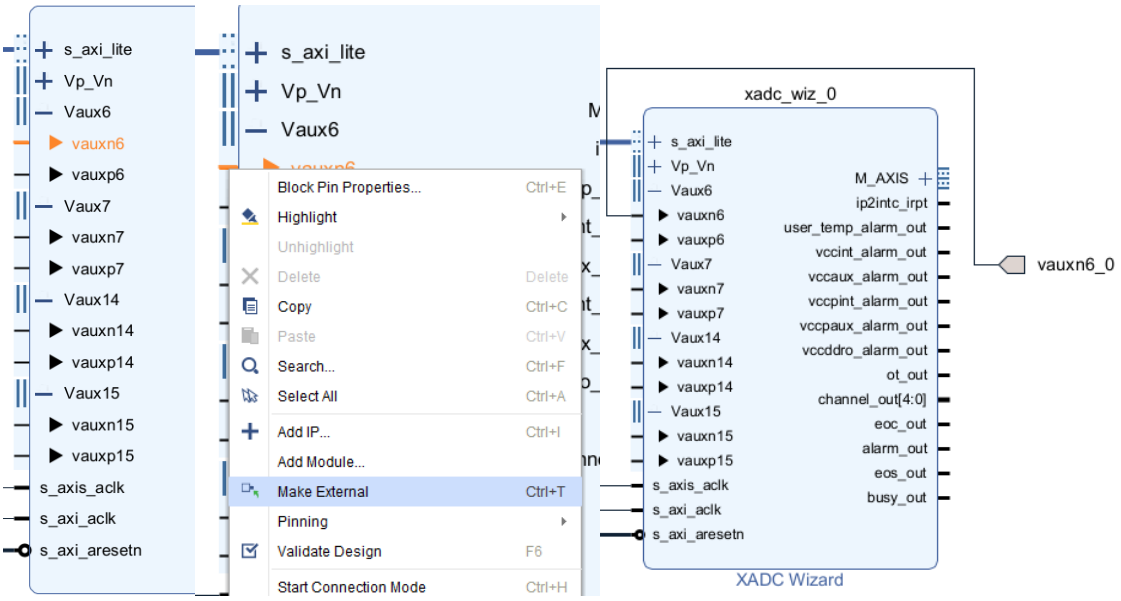

Figura 7.14. Selectarea pinului vauxn6 si configurarea setarea acestuia ca pin extern

După finalizarea acestui proces vom adăuga modulul de împărţire care foloseşte metoda cu refacerea restului parţial. Navigaţi la fereastra de surse, in partea stânga, unde este si designul vostru. Daca din orice motiv nu va mai apare aceasta fereastra in partea stânga a panelului de proiectare, o puteţi deschide navigând in meniul mediului Vivado, la opţiunea „**Window**" iar apoi selectând „**Sources**".

In meniul Sources, selectaţi sursa „**RestoringDivision**", iar apoi daţi **click dreapta** pe aceasta si apoi selectaţi opţiunea „**Add Module to Block Design**". Apăsaţi pe butonul „**Run Connection Automation**" pentru a lega ceasul modulului vhdl de ceasul sistemului de procesare. Selectaţi in fereastra care apare „**All Automation**" si apoi apăsaţi butonul „**Ok**".

Pentru a conecta pinii componentei de împărţire din vhdl la sistemul de procesare vom folosi module axi gpio. Conectarea modulelor vhdl folosind componente axi gpio a fost prezentata in detaliu in laboratoarele trecute, aşadar vom ilustra aceasta conectare pentru doi pini de intrare si doi de ieşire, pentru modulul adăugat, configurarea celorlalţi pini fiind realizata in mod analog (cu menţiunea ca va trebui setat numărul de biţi pentru fiecare modul axi gpio pentru a corespunde cu numărul de biţi ai pinului din componenta vhdl).

Apăsaţi pe butonul cu icoana „**+**" sau realizaţi combinaţia de taste **CTRL+I** pentru a include un nou IP, in fereastra care apare scrieţi in bara de căutare „AXI GPIO". Aceşti doi paşi sunt ilustraţi in Figura 7.15.

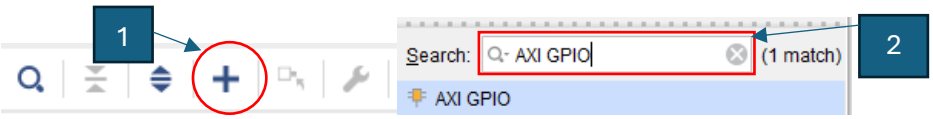

Figura 7.15. Deschiderea catalogului de IP-uri e ilustrat cu "1", căutarea componentei care dorim sa fie adăugată e prezentat cu "2"

Daţi dublu click pe componenta AXI GPIO care apare pentru a include o instanţa in proiectul dumneavoastră. După ce componenta a fost adăugată, daţi click pe aceasta si navigaţi la meniul „**Block Properties**", iar in tabul „**General**" la căsuţa corespunzătoare numelui redenumiţi componenta cu numele „axi_gpio_resetare" întrucât o vom folosi pentru a conecta procesorul la pinul de resetare al modulului vhdl. Apăsaţi tasta „**Enter**" după introducerea numelui astfel încât modificările sa se realizeze, iar apoi navigaţi la componenta adăugată si apăsaţi dublu click pe

aceasta pentru a o configura. Modificările pe care le vom realiza vor fi in tabul „**IP Configuration**", tabul „**Board**" poate rămâne cu configurația implicita. Bifați căsuța „**All Outputs**" si in căsuța corespunzătoare opțiunii „**GPIO Width**" introduceți valoarea 1 (fiindcă pinul de resetare este pe un bit), iar apoi apăsați butonul „**Ok**". Configurația este ilustrata in Figura 7.16. Setam modulul AXI GPIO ca si modul de ieșire, întrucât procesorul Zync **va furniza** valoarea pinului de resetare, care e pin de intrare in modulul vhdl.

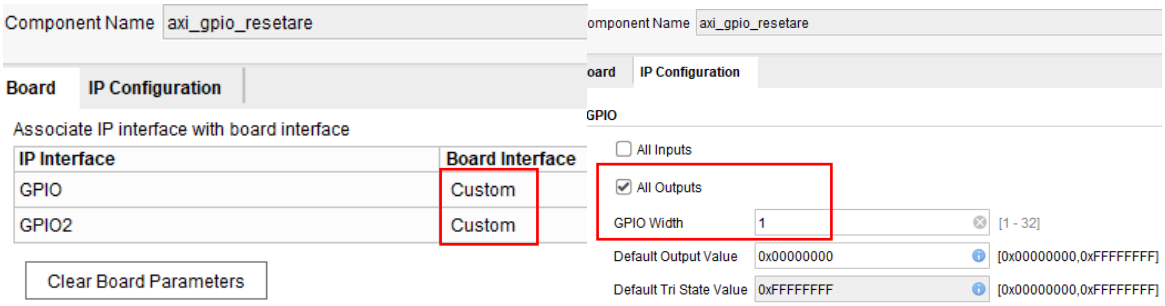

Figura 7.16. Configurația modulului AXI GPIO corespunzătoare pinului de resetare

Conectați pinul modulului AXI GPIO la portul resetare din vhdl, trăgând un fir intre cele doua porturi. Acest proces de conectare este explicat in detaliu in laboratorul 4.

Vom repeta procesul descris anterior si vom mai adaugă un modul AXI GPIO. Denumiți acest modul AXI GPIO cu numele „axi_gpio_dividend", iar după redenumire, dați **dublu click** pe acesta pentru a-l configura. La tabul „Board" va rămâne configurația implicita, iar in tabul „**IP Configuration**" vom bifa tabul „**All Outputs**" si setam GPIO Width la 12 întrucât operanzii noștri (dividendul si divizorul) sunt pe 12 biți. Configurația pentru acest bloc este ilustrata in Figura 7.17.

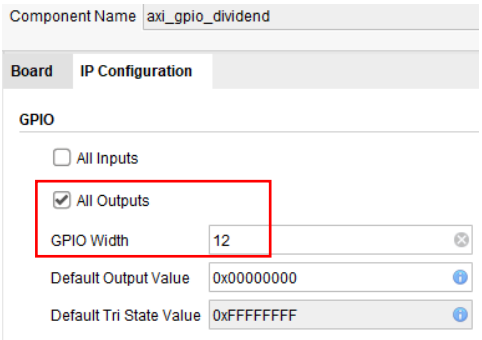

Figura 7.17. Configurația modulului axi gpio corespunzator pinului dividend

Realizați conexiunea intre pinii celor doua module (modulul axi_gpio_dividend si modulul de împărțire din vhdl).

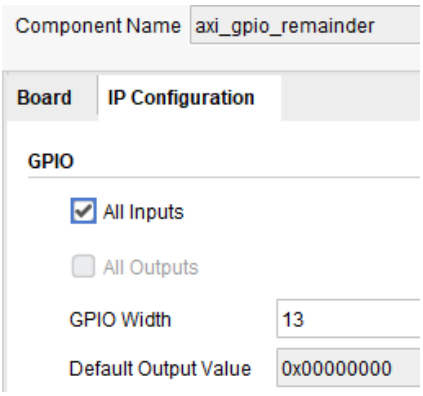

Figura 7.18. Configurarea modulului axi_gpio_remainder

Adăugați o noua componenta AXI GPIO, urmând pașii descrişi anterior si numiți aceasta componenta „axi_gpio_remainder”. Dați dublu click pe componenta adăugată, si in tabul „**IP Configuration**” bifați căsuța „**All Inputs**”, întrucât ieşirea din modulul vhdl va fi intrare in procesorul nostru, iar in căsuța text numita „**GPIO Width**” introduceți valoarea „**13**” întrucât restul împărţirii este pe N+1 biți, iar N in cazul nostru este 12. Configurația modulului axi_gpio_remainder este ilustrata in Figura 7.18. După configurare, apăsați butonul „**OK**” pentru a salva modificările făcute. Conectați modulul axi_gpio_remainder la pinul remainder din componenta vhdl.

Vom adăuga o noua componenta AXI GPIO, urmând pașii prezentați mai sus pentru portul „gata” de la modulul vhdl si vom numi aceasta componenta „axi_gpio_gata”. Dați **dublu click** pe componenta creata pentru a o configura. In tabul „IP Configuration” bifați căsuța „**All Inputs**” si setați „**GPIO Width**” la **1**, apăsați mai apoi butonul „**OK**” pentru a salva modificările făcute. Ca si la celelalte module AXI GPIO adăugate in cadrul acestui laborator, in tabul „Board” se vor lăsa opțiunile implicite. Configurarea din tabul IP Configuration pentru modulul axi_gpio_gata este ilustrata in Figura 7.19.

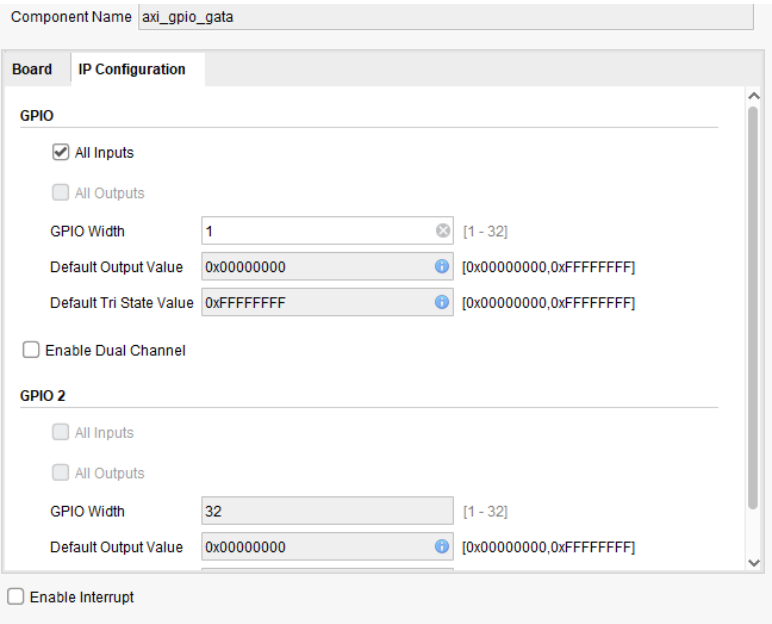

Figura 7.19. Configurarea modulului axi_gpio_gata

După configurare, conectați pinul de la modulul axi_gpio_gata la pinul gata al modulului de împărțire din vhdl. Schema de conectare la modulul de împărțire din vhdl, după adăugarea celor 4 componente AXI GPIO, arata ca in Figura 7.20.

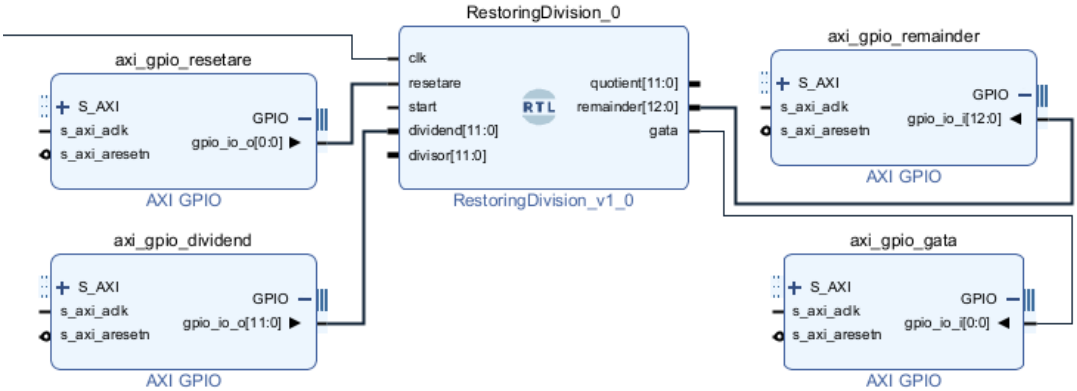

Figura 7.20. Schema de conectare a modulelor AXI GPIO la modulul de împărțire din vhdl

Adăugaţi, in mod similar cum a fost prezentat si anterior, module AXI GPIO pentru pinii ramaşi din modulul Restoring Division si configuraţi aceste module in mod corespunzător fiind atenţi la numărul de biţi ai fiecărui modul axi gpio si direcţia acestora (intrare sau ieşire). Pentru ca proiectarea hardware sa corespunda codului Vitis, care va fi oferit in acest laborator, numit modulele axi gpio ramase in felul următor: axi_gpio_start care va fi conectat la pinul de start, axi_gpio_divisor pentru pinul divisor si axi_gpio_quotient pentru pinul quotient. După adăugarea ultimelor 3 module axi gpio, apăsaţi butonul „**Run Connection Automation**", pentru ca Vivado sa conecteze modulele axi gpio la PS. In fereastra care va apărea, bifaţi „**All Automation"**, aşa cum e ilustrat si in Figura 7.21 si apoi apăsaţi butonul „**Ok**".

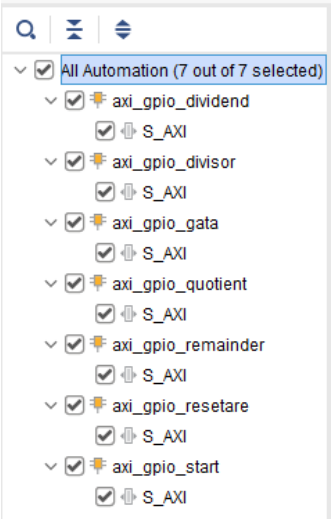

Figura 7.21. Selectarea tuturor pinilor modulelor AXI GPIO care vor fi conectaţi la PS.

In Figura 7.22 este ilustrata diagrama finala care include toate componentele necesare realizării proiectului. Apăsaţi pe simbolul ☑ pentru a valida designul creat, iar daca aţi urmat paşii descrişi in aceasta lucrare, va apărea mesajul „Validation Succesfull."

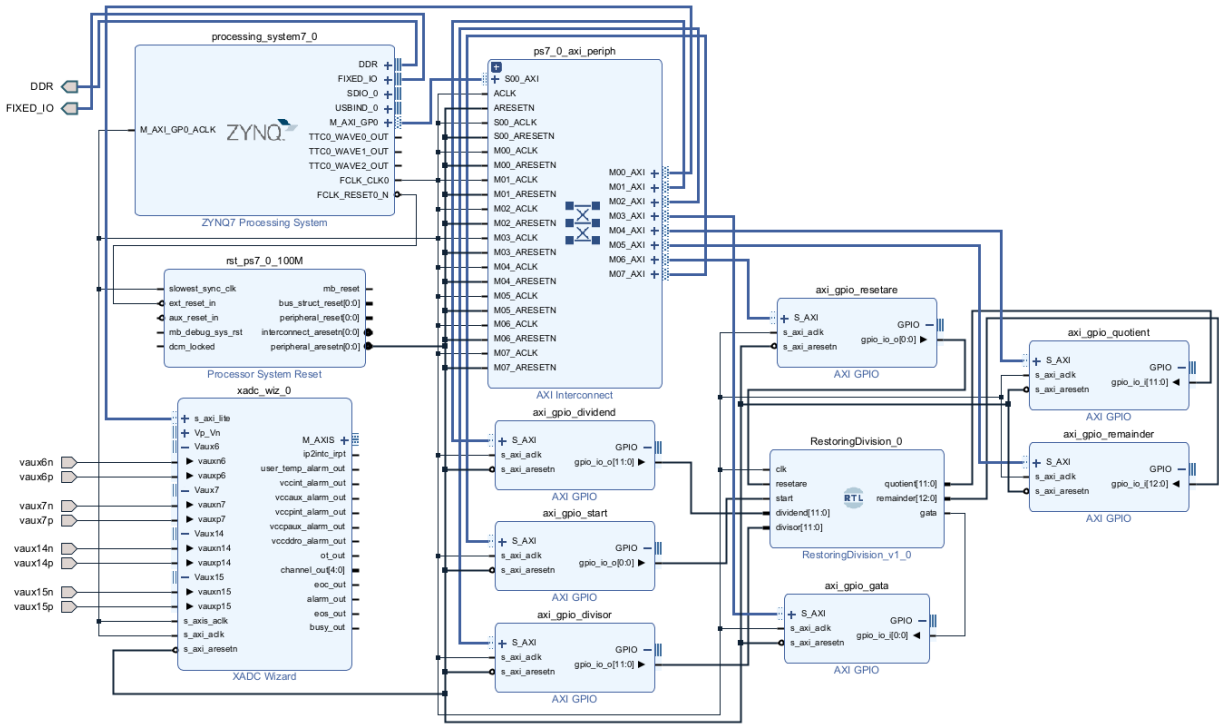

Figura 7.22. Diagrama finala a soluţiei proiectului

In secţiunea „**Sources**”, daţi **click dreapta** pe design_1 si selectaţi „**Create HDL Wrapper ...**”, apoi in fereastra care va apărea bifaţi opţiunea „**Let Vivado Manage Wrapper and auto-update**” si apăsaţi butonul „**Ok**”. După finalizarea creării invelitorii HDL apăsaţi pe butonul „**Generate Bitstream**”, din meniul Flow Navigator, iar in ferestrele care apar apăsaţi butoanele „**Yes**”, iar apoi „**OK**” pentru a genera fişierul *.bit al aplicaţiei. După finalizarea cu succes a generării fişierului *.bit, vom exporta hardware-ul navigând in meniul principal al mediului Vivado, la opţiunea File->Export->Export Hardware, apăsaţi butonul „**Next**” , bifaţi „**Include bitstream**”, apoi apăsaţi butonul „**Next**”, apoi din nou pe butonul „**Next**”, iar in final apăsaţi butonul „**Finish**”. In consola TCL va apărea mesajul „Successfully created Hardware Platform [...]” daca platforma a fost creata cu succes.

7.3.3. Implementarea in Vitis a aplicaţiei embedded

Deschideţi mediul Vitis 2024.1, dând dublu click pe icoana IDE-ului sau din meniul Vivado principal navigaţi la **Tools->Launch Vitis IDE**. Creaţi o platforma noua pe baza fişierului *.xsa exportat anterior. Acest pas este identic cu cei de la aplicaţiile din laboratoarele trecute, aşadar nu va fi prezentat in detaliu. Puteţi consulta laboratorul 3 secţiunea 2.2 daca nu va mai amintiţi cum sa efectuaţi acest pas.

Creaţi o aplicaţie in Vitis, cum a fost prezentat in lucrările trecute de laborator, pornind de la un exemplu „Hello World” si selectaţi platforma hardware generata anterior. Redenumiţi fişierul fişierul helloworld.c in Adc_project.c si introduceţi codul de mai jos.

```
#include <stdint.h> // Tipuri standard de date întregi
#include <stdio.h> // Funcții standard de input/output
//Funcții pentru inițializarea și curățarea platformei
#include "platform.h"
#include "xil_printf.h"// Funcții specifice Xilinx pentru afișare
#include "xil_types.h"// Definiții de tipuri specifice Xilinx
// Definiții pentru parametrii sistemului (adrese hardware, etc.)
#include "xparameters.h"
#include "xadcps.h"// API pentru XADC (convertor analogic-digital)
#include "xadcps_hw.h" // Definiții hardware specifice pentru XADC
#include <unistd.h> // Funcții pentru sleep (e.g., usleep)
#include "xgpio.h"// API pentru GPIO
#include "xil_exception.h"// Gestionarea excepțiilor
// Definirea ID-ului dispozitivului XADC utilizat
#define XADC_DEVICE_ID 0
// Numărul de biți cei mai puțin semnificativi ignorați la conversia
//datelor din XADC
#define LSB_BITS_NOT_USED 4
// Factorul de conversie pentru temperatura citită
#define TEMP_CONV_FACTOR 503.975/4096
// Termenul de conversie la grade celsius pentru temperatură
#define TEMP_CONV_TERM -273.15
// Factorul de conversie pentru citirile de tensiune
#define VOLT_CONV_FACTOR 3.0/4096
#define XADCPS_SEQ_CH_VAUX14 (1 << 30)// Mască pentru canalul VAUX14
#define XADCPS_SEQ_CH_VAUX7 (1 << 23)// Mască pentru canalul VAUX7
```

```c
#define XADCPS_SEQ_CH_VAUX15 (1 << 31)// Mască pentru canalul VAUX15
#define XADCPS_SEQ_CH_VAUX6 (1 << 22)// Mască pentru canalul VAUX6
// Pointer către structura de configurare a XADC
static XAdcPs_Config *xadc_cfg;
// Structura de control XADC pentru gestionarea convertorului ADC
static XAdcPs xadc_ctl;
// Instanțe GPIO pentru controlul operației de divizare: start,
//reset, dividend, divizor, cât, rest și semnal de finalizare.
XGpio startDvision,inputRst, dividend, divizor, outputDivQ,
outputDivR, outputGata;

int main()
{   // Inițializează resursele platformei (UART, timere, etc.)
    init_platform();
    // Variabilă pentru a stoca starea apelurilor funcțiilor
    s32 status;
    // Variabile pentru a stoca citirile de la XADC
    u16 temp, vccint, vccaux, vbram, vccpint, vccpaux, vccpdro;
    // Factor de scalare pentru divizare
    uint16_t factorScalareConstant = 14;
    //Inițializează GPIO-urile pentru comunicarea cu modulului din PL
    XGpio_Initialize(&startDvision, XPAR_AXI_GPIO_START_BASEADDR);
    XGpio_Initialize(&inputRst, XPAR_AXI_GPIO_RESETARE_BASEADDR);
    XGpio_Initialize(&dividend, XPAR_AXI_GPIO_DIVIDEND_BASEADDR);
    XGpio_Initialize(&divizor, XPAR_AXI_GPIO_DIVISOR_BASEADDR);
    XGpio_Initialize(&outputDivQ, XPAR_AXI_GPIO_QUOTIENT_BASEADDR);
    XGpio_Initialize(&outputDivR, XPAR_AXI_GPIO_REMAINDER_BASEADDR);
    XGpio_Initialize(&outputGata, XPAR_AXI_GPIO_GATA_BASEADDR);
    // Setează direcția GPIO-urilor, 0 - intrare, 1-iesire
    XGpio_SetDataDirection(&startDvision, 1, 0);// intrare
    XGpio_SetDataDirection(&inputRst, 1, 0);//intrare
    XGpio_SetDataDirection(&dividend, 1, 0x000);//intrare
    XGpio_SetDataDirection(&divizor, 1, 0x000);//intrare
    XGpio_SetDataDirection(&outputDivQ, 1, 0xFFF);//iesire
    XGpio_SetDataDirection(&outputDivR, 1, 0b1111111111111);//iesire
    XGpio_SetDataDirection(&outputGata, 1, 1);//iesire
    // Caută configurația XADC după ID-ul dispozitivului
    xadc_cfg = XAdcPs_LookupConfig(XADC_DEVICE_ID);
    if(NULL == xadc_cfg)
    {   // Afișează un mesaj de eroare dacă nu s-a găsit configurația
         print("Configuratia a crapat \n\r");
         return XST_FAILURE; // Returnează starea de eșec
    }
// Inițializează XADC cu configurația găsită.
    status = XAdcPs_CfgInitialize(&xadc_ctl, xadc_cfg, xadc_cfg->
BaseAddress);
```

```
    if(status != XST_SUCCESS){
  // Afișează un mesaj de eroare dacă inițializarea XADC a eșuat.
        print("Statusul a crapat \n\r");
        return XST_FAILURE; // Returnează starea de eșec.
    }
   // Afișează un mesaj de succes după configurarea reușită
    print("Configuratia a mers bine, incepe procesarea \n\r");
// Setează XADC în modul de conversie continuă.
 XAdcPs_SetSequencerMode(&xadc_ctl,XADCPS_SEQ_MODE_CONTINPASS);
// Creează o mască pentru canalele VAUX14, VAUX7, VAUX15 și VAUX6
    u32 ChannelMask = XADCPS_SEQ_CH_VAUX14 | XADCPS_SEQ_CH_VAUX7 |
XADCPS_SEQ_CH_VAUX15 | XADCPS_SEQ_CH_VAUX6;
// Activează canalele specificate în XADC
    XAdcPs_SetSeqChEnables(&xadc_ctl, ChannelMask);
while (1) {
    // Citește parametrii de la XADC(temperature, tensiune etc.)
    temp = XAdcPs_GetAdcData(&xadc_ctl,XADCPS_CH_TEMP);
    vccint =  XAdcPs_GetAdcData(&xadc_ctl,XADCPS_CH_VCCINT);
    vccaux =  XAdcPs_GetAdcData(&xadc_ctl,XADCPS_CH_VCCAUX);
    vbram =  XAdcPs_GetAdcData(&xadc_ctl,XADCPS_CH_VBRAM);
    vccpint =  XAdcPs_GetAdcData(&xadc_ctl,XADCPS_CH_VCCPINT);
    vccpaux = XAdcPs_GetAdcData(&xadc_ctl,XADCPS_CH_VCCPAUX);
    vccpdro = XAdcPs_GetAdcData(&xadc_ctl,XADCPS_CH_VCCPDRO);
    // Convertește și afișează citirile de temperatură și tensiune
    printf("Die temperature %.2f C \n",((float)(temp >>
LSB_BITS_NOT_USED)) * TEMP_CONV_FACTOR + TEMP_CONV_TERM);
    printf("VCCINT: %.3f V \n", ((float)(vccint >>
LSB_BITS_NOT_USED)) * VOLT_CONV_FACTOR);
    printf("VCCAUX: %.3f V \n", ((float)(vccaux >>
LSB_BITS_NOT_USED)) * VOLT_CONV_FACTOR);
    printf("VBRAM: %.3f V \n", ((float)(vbram >> LSB_BITS_NOT_USED))
* VOLT_CONV_FACTOR);
    printf("VCCPINT: %.3f V \n", ((float)(vccpint >>
LSB_BITS_NOT_USED)) * VOLT_CONV_FACTOR);
    printf("VCCPAUX: %.3f V \n", ((float)(vccpaux >>
LSB_BITS_NOT_USED)) * VOLT_CONV_FACTOR);
    printf("VCCPDRO: %.3f V \n", ((float)(vccpdro >>
LSB_BITS_NOT_USED)) * VOLT_CONV_FACTOR);
   // Citește datele pentru canalele auxiliare VAUX
   //desi citim datele de la toate cele 4 canale, noi potentiometrul
   //il vom lega doar la un canal, restul vor ramane in aer
   //lasam totusi citirea de la toate canalele pentru a demonstra
   //cum se realizeaza un astfel de proces
    unsigned long Vaux14Data = XAdcPs_GetAdcData(&xadc_ctl, 30);
    unsigned long Vaux7Data = XAdcPs_GetAdcData(&xadc_ctl, 23);
    unsigned long Vaux15Data = XAdcPs_GetAdcData(&xadc_ctl, 31);
    unsigned long Vaux6Data = XAdcPs_GetAdcData(&xadc_ctl, 22);
   Vaux14Data >>= LSB_BITS_NOT_USED; //luam 12 biti
```

```
//afisam datele citite
printf("Potentiometer Value: %lu %lu %lu %lu \r\n" ,Vaux14Data,
Vaux7Data,Vaux15Data, Vaux6Data);
// Scriem valorile necesare executarii operatiei de impartire pe
pinii
//corespunzatori, pentru a fi communicate modulului din vhdl
    XGpio_DiscreteWrite(&inputRst, 1, 0);
    XGpio_DiscreteWrite(&dividend, 1, Vaux14Data);
    XGpio_DiscreteWrite(&divizor, 1, factorScalareConstant);
   // Pornire operație de divizare
    XGpio_DiscreteWrite(&startDvision, 1, 1);
//asteptam pana cand modulul vhdl ne semnalizeaza ca operatia e
//finalizata, adica semnalul gata devine 1
 int gataImpartire = 0;
  do{
   gataImpartire = XGpio_DiscreteRead(&outputGata, 1);
  }while(gataImpartire == 0); //facem pooling
//După ce semnalul gataImpartire devine 1, adăugăm o mică întârziere
// de 1 milisecundă (usleep(1000))
//pentru a ne asigura că semnalele ieșirii sunt complet stabilizate
și
// propagate
    usleep(1000);
    // Citește câtul și restul din GPIO
    int quotient = XGpio_DiscreteRead(&outputDivQ, 1);
    int remainder = XGpio_DiscreteRead(&outputDivR, 1);
    // Afișează câtul și restul.
    printf("Catul Obtinut %d, rest obtinut %d \n",quotient,
remainder);
    // Resetează semnalul de start divizare
    XGpio_DiscreteWrite(&startDvision, 1, 0);
    // Pauză de 1 secunda înainte de a relua bucla.
    usleep(1000000);
    }
    // Curăță resursele platformei înainte de a ieși
    cleanup_platform();
    // Returnează starea de finalizare cu succes
    return 0;
}
```

În general, utilizarea unui ADC auxiliar într-un proiect pe platforma Vitis implică mai mulți pași importanți pentru a asigura corecta configurare și funcționare a canalelor ADC. În primul rând, este necesară obținerea unei configurații hardware corecte asociate dispozitivului XADC din sistem. Acest lucru se face prin utilizarea funcției XAdcPs_LookupConfig(), care caută configurația predefinită a dispozitivului ADC asociat cu un anumit ID hardware. În exemplul dat, XADC_DEVICE_ID este o constantă care identifică instanța XADC utilizată în sistemul descris, iar această constantă este definită de obicei în fișierele de configurație generate de platforma Vitis sau de Vivado. Apoi este necesar să fie inițializat driverul corespunzător pentru XADC (Xilinx

Analog-to-Digital Converter). Aceasta se face prin apelarea funcţiilor de iniţializare, cum ar fi XAdcPs_CfgInitialize(). Funcţia aceasta pregăteşte ADC-ul şi asigură că toate resursele hardware şi software sunt disponibile pentru operaţiile viitoare. După iniţializarea ADC-ului, se pot seta canalele pe care dorim să le utilizăm în procesul de conversie. Selectarea canalelor implică utilizarea registrelor de secvenţiere din XADC. Fiecare canal, fie el dedicat tensiunii de alimentare, temperaturii sau unui semnal analogic extern (VAUX), are o poziţie specifică în registrul secvenţei de conversie. Această poziţie este documentată în manualul Xilinx [3], care detaliază arhitectura şi specificaţiile pentru Zynq. De exemplu, pentru canalul VAUX14, poziţia în registrul de secvenţiere este bitul 30, ceea ce înseamnă că pentru a activa acest canal, trebuie să fie setat bitul 30 la valoarea 1. În mod similar, canalul VAUX7 este mapat pe bitul 23. După identificarea corectă a canalelor pe care dorim să le folosim, acestea pot fi activate prin setarea corespunzătoare a acestor biţi în registrul de secvenţiere folosind funcţia XAdcPs_SetSeqChEnables(). Citirea valorilor se face prin funcţia XAdcPs_GetAdcData(), care returnează datele brute din registrul ADC pentru canalul respectiv. Aceste date trebuie convertite la o valoare interpretabila folosind o constantă de conversie, de regula oferita de foaia de date a senzorului utilizat.
Aşadar procesul general pentru configurarea unui ADC auxiliar în Vitis implică:

1. Obţinerea unei configuraţii hardware corecte prin XAdcPs_LookupConfig(XADC_DEVICE_ID).
2. Iniţializarea driverului ADC folosind funcţia XAdcPs_CfgInitialize().
3. Configurarea canalelor dorite prin funcţii precum XAdcPs_SetSeqChEnables() şi activarea lor folosind măşti de biţi, cum ar fi XADCPS_SEQ_CH_VAUX14 pentru VAUX14
4. Citirea datelor brute folosind funcţia XAdcPs_GetAdcData() si scalarea acestora pe baza informaţiei provenite de foaia de date a senzorului.

7.3.4 Rularea aplicaţiei pe plăcuţa Zybo

Un convertor ADC de pe placa Zybo are o impedanţă de intrare foarte mare, de ordinul megohmilor. Acest aspect este important deoarece, la măsurarea semnalelor, nu dorim ca aparatul de măsură (cum ar fi un voltmetru) să influenţeze circuitul pe care îl măsurăm. În cazul unui voltmetru, este crucial ca rezistenţa sa internă să fie foarte mare, astfel încât să nu "simtă" circuitul prezenţa voltmetrului. Practic, voltmetrul trebuie să aibă o rezistenţă ridicată pentru a evita modificarea caracteristicilor circuitului măsurat. Pentru a nu avaria convertorul analog digital de pe placa Zybo, trebuie sa ne asiguram ca tensiunea de intrare in placa a senzorului sau potenţiometrului, este de maxim un volt. Pentru a realiza acest lucru va trebui sa folosim un divizor de tensiune, si sa găsim valoarea rezistentei care va trebui adăugată circuitului nostru pentru a obţine o tensiune de ieşire de 1V. Conform legii lui Ohm, cunoaştem ca intr-un circuit simplu, curentul I este egal cu tensiunea totala U_{total} împărţita la rezistenta totala R_{total} din circuit. In cazul nostru rezistenta totala este egala cu suma rezistentelor individuale întrucât rezistentele sunt in serie. Înlocuind U_{total}= 3.3V si $R_{total} = R_X + R_p$ unde R_p reprezintă rezistenta potenţiometrului iar R_x este rezistenta pe care dorim sa o calculam, obţinem ecuaţia (7.1):

$$I = \frac{U_{total}}{R_{total}} = \frac{3.3V}{Rx+Rp} \tag{7.1}$$

De asemenea, cunoaştem ca tensiunea din ieşirea potenţiometrului este exprimata conform ecuaţiei (7.2).

$$U_p = I * R_p \tag{7.2}$$

Cunoscând ca tensiunea potenţiometrului trebuie sa fie de maxim 1V, rezistenta potenţiometrului achiziţionat este de 10k si formula (7.1), deducem valoarea rezistentei utilizate in divizorul de tensiune conform (7.3).

$$U_p = \frac{3.3V * R_p}{R_x + R_p} \; adica \; 1V = \frac{3.3V * 10K\Omega}{R_x + 10K\Omega} \tag{7.3}$$

Aşadar valoarea rezistentei noastre ideale R_x este de 23KΩ. In practica vom folosi o rezistenta de 22 KΩ întrucât acestea sunt mult mai frecvente, iar variaţia in tensiunea de ieşire este neglijabila in cazul utilizării unei astfel de rezistente. Diagrama de conectare a potenţiometrului la intrările diferenţiale ale XADC-ului este ilustrata in Figura 7.23.

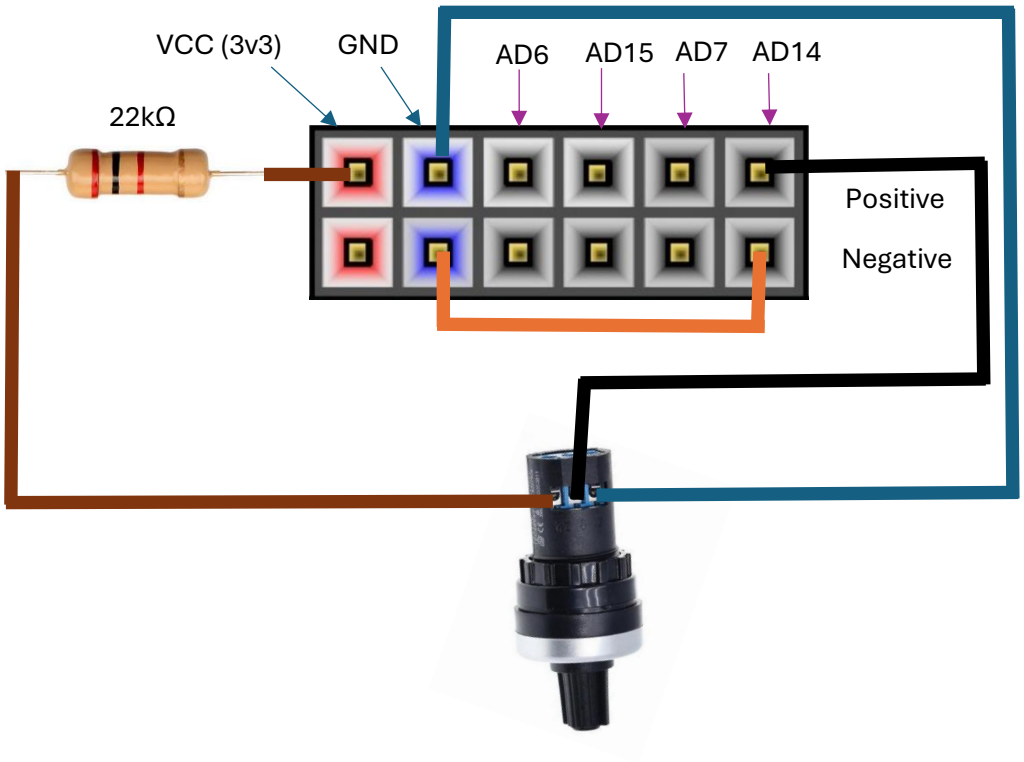

Figura 7.23. Schema de conectare a potenţiometrului la intrările diferenţiale ale XADC-ului

Schema de conectare a potenţiometrului, si calcului rezistentei folosite in divizorul de tensiune au fost explicate pentru a înţelege in profunzime modul de lucru a potenţiometrului cu XADC-urile de pe plăcută Zybo. Studenţii vor primi potenţiometrul împreuna cu divizorul de tensiune lipite pe o bareta de pini care trebuie doar sa fie conectata la XADC-ul plăcutei.

Figura 7.24. Placă Zybo cu potenţiometru şi afişarea tensiunilor citite de la XADC-uri

In Figura 7.24. afişam in HTerm toate valorile analogice, menţionate la începutul secţiunii 7.3, citite prin intermediul XADC-urilor. Configuraţia programului HTerm este la fel ca si in laboratoarele tracute. De asemenea, observam si rezultatul împărţirii valorii achiziţionate de la pinul AD14 folosind factorul de scalare constant setat in codul C (888 : 14 = 63 rest 6). Valorile 239, 811 si 210 sunt obţinute prin citirea de la pinii lăsaţi in aer (AD7, AD15, AD6). Achiziţia de la aceşti pini a fost realizata doar pentru a demonstra modul in care se pot citii date de la mai multe porturi XADC in Vitis, fără a conecta efectiv un senzor la pinii menţionaţi.

Codul C din Vitis si proiectarea hardware din Vivado pot fi găsite la linkul de github de mai jos: https://github.com/mirceamp/StructuraSistemelorDeCalcul/tree/main/Lab7

Exerciţii

1. Implementaţi si rulaţi tutorialul prezentat in lucrarea de laborator pentru a citii datele de la senzorii interni plăcutei Zybo si de la potenţiometru.
2. Modificaţi codul astfel încât verificarea folosind pooling sa fie înlocuita de o întrerupere externa ataşata pinului gata din algoritmul de împărţire. (Indiciu: uitaţi-va la laboratorul cu întreruperi externe).

Tema

3. Implementaţi in vhdl metoda de împărţire fără refacerea restului parţial. Verificaţi corectitudinea implementării folosind un test bench si apoi înlocuiţi metoda bazata pe refacerea restului parţial, din tutorialul de mai sus, cu noua metoda de împărţire implementata de voi.

Referinţe

[1] AMD, „7 Series FPGAs and Zynq-7000 SoC XADC Dual 12-Bit 1 MSPS Analog-to-Digital Converter User Guide (UG480)", Sept. 2022. *[Online]. Available*: https://docs.amd.com/r/en-US/ug480_7Series_XADC/XADC-Overview *[Accessed: 20-Oct-2024]*

[2] Konrad, A., Scholl, C., Mahzoon, A., Große, D., and Drechsler, R., "Divider verification using symbolic computer algebra and delayed don't care optimization: theory and practical implementation," *Formal Methods in System Design*, vol. 57, no. 3, pp. 678-693, May 2024. DOI: 10.1007/s10703-024-00452-3.

[3] *AMD, "7 Series FPGAs and Zynq-7000 SoC XADC User Guide," Dec. 2022. [Online]. Available: https://docs.amd.com/r/en-US/ug480_7Series_XADC/Sequencer-Modes. [Accessed: 20-Oct-2024]*